

大模型原生计算体系结构

基于计算机体系结构类比的模型原生系统架构构想

Model-Native Computing Architecture: Envisioning Model-Native System
Architecture Through the Lens of Computer Architecture

林海* 詹少雄†

清华大学深圳国际研究生院
鹏城实验室，深圳

2026年5月

Abstract

大语言模型正在从一种"模型技术"演变为一种"系统技术"。当我们在用Codex写代码、用Claude Code管理项目、用AutoGPT执行多步任务时，围绕大模型的工程问题——缓存复用效率、上下文容量管理、Agent调度与权限控制——越来越像经典计算机系统问题。一个自然的想法浮现：如果把大模型看作CPU，把KV缓存看作高速缓存，把上下文窗口看作内存，把Agent框架看作操作系统，那么过去八十年计算机体系结构积累的分层抽象与设计原则，能否帮助我们理解和构建未来的模型原生计算系统？

本文正是沿着这一思路展开的一篇技术畅想型综述。我们首先用通俗的语言建立了计算机体系结构与大模型系统之间的类比映射，然后系统梳理了LLM-as-OS、记忆管理、Agent框架、工具协议、多Agent协作、认知架构与安全治理等方向的相关研究，指出它们各自覆盖了模型原生系统栈的不同层次，但缺少统一的分层模型。为填补这一空白，本文提出**智能计算体系结构模型（ICAM）**，将模型原生计算系统划分为从硬件执行到用户应用的六个功能层，每层都有明确的接口契约和设计公理。同时，我们揭示了现有文献中一个核心分歧——大模型到底更像CPU还是操作系统——并提出**双平面架构**作为统一解答：模型原生计算系统实际上同时包含一个概率执行平面（负责"能做什么"）和一个确定性控制平面（负责"应该做什么"），不同文献看到的只是不同平面的不同切面。

在量化层面，本文提出三条设计定律：语义局部性定律揭示KV缓存命中率与推理加速的关系；上下文预算定律指出有效工作集受窗口容量和注意力衰减的双重约束；Agent加速比定律说明多Agent协作的收益递减规律。三条定律均用已发表系统数据进行了实证检验。最后，本文总结类比的适用边界与本质差异，给出面向未来的研究路线图。全文为综述与概念性分析，不包含新实验。

关键词： 大模型系统；计算机体系结构；智能计算体系结构框架；量化定律；KV缓存；上下文工程；Agents

1 引言：模型原生计算需要体系结构

大语言模型正在从一种"模型技术"演变为一种"系统技术"。从GPT-3展示少样本学习能力[?]，到LLaMA系列推动开放模型生态[?]，再到工具使用[?]、检索增强[?]、自主Agent[?]和代码生成系统[?, ?]的快速涌现，研究重心已经从"如何训练更强的模型"，转向"如何把智能组织成稳定、可扩展、可审计的系统"。这一转变同时正在重塑工程实践本身：工程师的角色正从"逐行编写代码"转向"编排管理多个Agent系统"，整个开发生命周期正在从周级别压缩到小时级

*ngyygm@outlook.com

†zhansx24@mails.tsinghua.edu.cn

别[?]。然而，当前实践中的许多核心挑战—内存带宽受限[?]、缓存复用效率[?]、上下文管理[?]、批处理调度[?]、沙箱与权限控制[?]-并非模型本身的能力问题，而是典型的系统问题。

这种系统性转变已在工业实践中产生显著效果。Rakuten的工程师使用Claude Code在包含1250万行代码的vLLM项目中自主完成了复杂的激活向量提取任务，耗时7小时，数值精度达99.9%；Augment Code的一位企业客户将CTO预估需要4-8个月的项目在2周内完成[?]。这些案例表明，模型原生系统面临的挑战已经从理论可能性变为工程现实。

面对这些问题，一条自然的思路浮现：经典计算机体系结构曾用一套成熟的分层抽象解决了类似的复杂性管理问题，那么这套思想能否迁移到模型原生计算系统？

1.1 一个直接的类比

这个类比的核心映射关系可以用一张表来说明：

经典计算	模型原生计算	共同的本质问题
CPU（通用处理器）	LLM推理核心	通用计算/推理能力
高速缓存（L1/L2/L3）	KV缓存	热数据复用与访存优化
内存& 虚拟内存	上下文窗口& 外部记忆	有限容量的地址空间管理
操作系统	Agent运行时	调度、权限、资源管理
I/O总线（PCIe/USB）	MCP/A2A协议	标准化的外设/工具接入
应用程序与用户	Agent应用与领域专家用户	将计算能力转化为领域解决方案

具体而言：（1）大语言模型的推理核心承担通用的语义理解与生成能力，类似于CPU执行通用指令集—不同的模型（如GPT-4、Claude、LLaMA）就像不同的CPU微架构，执行同样的"推理任务"但性能特征各异。（2）KV缓存（Key-Value Cache）在自回归解码过程中复用先前计算过的注意力键值对，避免重复计算，这与CPU高速缓存利用时间局部性复用热点数据的原理完全一致；PagedAttention[?]更是直接借鉴了操作系统分页管理的思想，将KV缓存划分为固定大小的"页"进行按需分配。（3）上下文窗口容量有限（如128K token），但Agent需要访问远超此限制的知识和历史交互，这与物理内存有限但程序需要更大地址空间的问题如出一辙；MemGPT[?]因此将"虚拟上下文管理"作为核心抽象，类比操作系统的虚拟内存机制。（4）Codex[?]、Claude Code[?]、OpenHands[?]等Agent运行时负责任务分解、工具调度、子Agent管理、沙箱隔离与结果提交，其职责范围越来越接近一个操作系统内核。（5）MCP（Model Context Protocol）[?]定义了Agent与外部工具之间的标准化接口，A2A（Agent-to-Agent Protocol）[?]定义了Agent之间的互操作协议，它们共同扮演着"I/O总线"的角色—MCP类似于垂直总线（如PCIe，连接Agent与工具），A2A类似于水平总线（如网络互连，连接Agent与Agent）[?]。

2023年，Andrej Karpathy用一句简短的话概括了这一直觉："LLMs are not chatbots, they are the kernel process of a new Operating System"[?]。这一观察之所以引发广泛共鸣，正是因为它抓住了正在发生的事实：围绕大模型的工程问题，越来越像经典计算机系统问题。

图 ??给出了计算机体系结构与模型原生计算系统之间的类比映射总览。

1.2 从直觉到问题

然而，类比本身并不等于理论。"像"不等于"是"。如果类比仅停留在修辞层面，它只能产生启发性的比喻，不能产生可验证的预测和可操作的设计原则。

近年来，已有多个独立的工作从不同角度触及了这一类比。AIOS将LLM视为操作系统内核，构建了包含调度器、上下文管理器和内存管理器的Agent运行时[?]。MemGPT用虚拟内存思想管理上下文，提出了"虚拟上下文管理"的抽象[?]。Mi等人从计算机系统演进的角度设计Agent架构，借鉴进程、文件系统等经典概念[?]。Ge等人提出"LLM as OS, Agents as Apps"的愿景[?]。L2MAC将LLM框架称为stored-program automatic

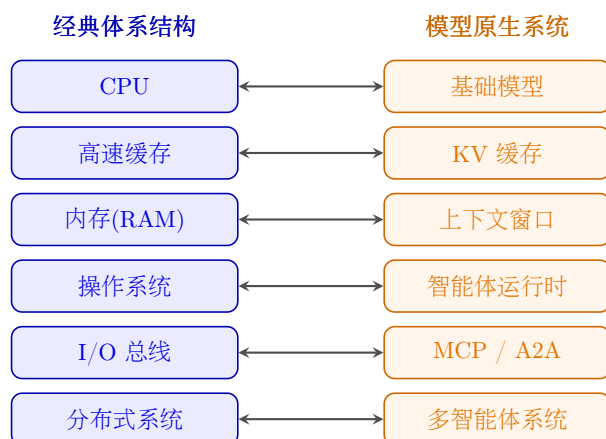


Figure 1: 计算机体系结构与模型原生计算系统的类比映射总览

computer[?]. ArbiterOS把LLM定义为受governor治理的"Probabilistic CPU"[?]. AgentOS将其视为reasoning kernel[?]. MemOS将记忆本身作为操作系统级的一等资源进行管理[?]. 在推理工程侧, Mei等人于2025年系统调研了上下文工程 (Context Engineering) 这一新兴学科, 将其定义为"系统性地优化输入给LLM的全部信息"的工程实践[?].

这些工作各覆盖了系统栈中的一层或几层—OS层、内存层、Agent内核层、推理层。但它们之间存在根本性的隐喻冲突: 模型到底更像CPU、OS kernel、还是reasoning kernel? 更关键的是, 它们之间缺少统一的分层模型、层间接口定义和可量化的设计原则。此外, 这些工作几乎都将人类视为系统的边界条件, 而非交互循环的核心组件。然而最新的工业实证表明, 人类判断力与Agent执行能力之间的协作模式—而非Agent的独立能力—才是决定系统效率的关键因素: 工程师在工作中约60%使用AI, 但只能"完全委托"0-20%的任务[?]. 就像在计算机体系结构的早期, 如果没有ISA (指令集架构) 这一层作为软硬件之间的契约, 每一代新CPU都需要重写所有软件, 每一层改进都无法被其他层复用。

1.3 本文的贡献

因此, 本文尝试做以下五件事。

其一, 回到计算机体系结构、操作系统与分布式系统的基本抽象, 提炼出适合模型原生系统栈的类比框架, 明确类比的适用范围与局限。

其二, 系统梳理相关研究工作, 揭示现有隐喻之间的冲突, 提出**双平面架构**作为统一解答—将模型原生系统栈划分为"概率平面" (模型推理与生成) 和"确定平面" (系统调度与控制), 两平面通过明确定义的接口交互。

其三, 在此基础上提出**智能计算体系结构模型 (ICAM)**—一个六层分层框架, 包含层间接口契约与六条设计公理。

其四, 提出三条**量化定律**, 为模型原生计算系统提供类似Amdahl定律和Roofline模型对传统计算机体系结构那样的可计算原则, 并用已发表数据做实证检验。

其五, 总结当前开源实现和工业实践, 给出一个面向未来的研究路线图。

2 背景: 经典计算栈与模型原生系统栈

本节先回顾经典计算机体系结构的分层逻辑及其背后的设计原则, 再梳理当前大模型系统中正在自然出现的分层雏形, 为后续的类比框架奠定基础。

2.1 经典计算机体系结构的分层逻辑

传统计算机体系结构的分层并非偶然的设计选择，而是处理复杂性的通用工程方法[?]。其核心思想是：每一层只依赖其下层的接口（interface），而不依赖其实现（implementation）。这使得任何一层可以在不改变接口的前提下独立优化，而无需通知其他层。以下逐一介绍各层的核心理念。

2.1.1 指令集架构（ISA）：软硬件之间的契约

指令集架构（Instruction Set Architecture, ISA）定义了软件可见的机器行为规范：有哪些指令（算术、访存、分支、跳转）、有多少寄存器、异常如何触发和处理、地址空间如何组织。ISA的重要性在于它是一份"契约"—只要CPU遵守这份契约，上层软件就可以正确运行，完全不需要知道CPU内部是如何实现的。Intel x86、ARM和RISC-V代表了三种截然不同的ISA设计哲学[?, ?]: x86追求向后兼容性（40年前的代码至今仍可运行），ARM追求能效比（在移动设备上表现优异），RISC-V追求开放与模块化（允许用户自定义扩展指令）。但它们服务于同一个目的：给上层软件一个稳定的编程目标。

2.1.2 微架构：同一契约下的不同实现

微架构（Microarchitecture）决定了同一个ISA下CPU的具体性能表现。它回答的问题是：一条指令到底需要几个时钟周期完成？可以同时执行多少条指令？猜错的分支要付出什么代价？具体来说，流水线（pipeline）将指令的执行分为取指、译码、执行、访存、写回等多个阶段，让不同指令的不同阶段可以重叠执行；超标量（superscalar）在每个时钟周期发射多条指令到不同的执行单元；分支预测器（branch predictor）根据历史模式猜测条件分支的走向，避免流水线气泡。同一个x86 ISA可以由Intel的Core微架构和AMD的Zen微架构分别实现，性能各异，但对软件透明。

2.1.3 缓存层次：利用局部性加速数据访问

缓存层次（Cache Hierarchy）解决的核心问题是CPU与主存之间的速度差距。CPU可以在1纳秒内完成一次操作，而访问DRAM需要约100纳秒—如果CPU每次需要数据都要等DRAM，性能将下降两个数量级。缓存的解决方案基于局部性原理：时间局部性（temporal locality）指刚被访问的数据很可能很快再次被访问，空间局部性（spatial locality）指地址相邻的数据很可能很快被访问[?]。因此，缓存把最近和附近的数据保存在速度更快但容量更小的SRAM中：L1缓存（约32–64 KB，4个周期延迟）最接近CPU核心，L2缓存（约256 KB–1 MB，约10个周期延迟）作为二级缓冲，L3缓存（数MB至数十MB，约40个周期延迟）被多核共享。当缓存未命中（cache miss）时，数据才需要从DRAM加载，代价约为200个周期。这种分层设计用SRAM的速度服务频繁访问，用DRAM的容量服务整体需求。

2.1.4 虚拟内存：有限的物理资源，无限的地址幻觉

虚拟内存（Virtual Memory）通过内存管理单元（MMU）和页表（page table），给每个进程提供一个独立的、远大于物理内存的地址空间。其核心机制是分页（paging）：将虚拟地址空间划分为固定大小的"页"（通常4 KB），将物理内存划分为同样大小的"页框"（page frame），通过页表记录"虚拟页号→物理页框号"的映射。当进程访问一个尚未映射到物理内存的虚拟页时，触发缺页异常（page fault），操作系统将该页从磁盘加载到物理内存，然后恢复进程执行[?]。这样，每个进程都以为自己拥有完整的地址空间，而操作系统在幕后管理着有限物理资源的分配和回收。虚拟内存还提供了进程间隔离：每个进程的页表独立，无法访问其他进程的内存。

2.1.5 操作系统：调度、保护与资源管理

操作系统（Operating System）是硬件之上的第一层软件，负责管理计算资源并提供统一的抽象。它的核心职责包括：进程调度（deciding which process runs next, e.g., Linux CFS[?]）、内存管理（分配和回收物理页框）、I/O管理（提供统一的文件和设备接口）、权限控制（确保进程只能访问被授权的资源）和并发支持（信号量、锁、条件变量）[?, ?]。操作系统的设计原则是：向上提供简洁的抽象（进程、文件、套接字），向下管理复杂的硬件（CPU核心、内存条、磁盘、网卡）。用户程序只需调用`open()`、`read()`、`fork()`等系统调用，无需关心磁盘是SSD还是HDD、网卡是万兆还是千兆。

2.1.6 分布式系统：从单节点到多节点的扩展

分布式系统（Distributed Systems）进一步把多个节点组织成一个逻辑上统一的整体。它需要解决的核心问题是：当部分节点故障、网络延迟不确定、数据需要复制时，如何保证系统的正确性和可用性[?]。Raft共识算法[?]让多个节点就"日志中下一条命令是什么"达成一致，即使部分节点崩溃也不影响整体正确性。Google Spanner[?]在全球多个数据中心之间提供强一致性的分布式数据库服务。CAP定理[?]则指出，在网络分区（Partition）发生时，系统无法同时保证一致性（Consistency）和可用性（Availability），必须在两者之间做出权衡。

2.1.7 可计算的设计原则

更关键的是，经典体系结构不仅有分层抽象，还有**可计算的原则**（quantitative principles）。这些原则使体系结构从"经验工程"上升为"定量科学"。

Amdahl定律告诉我们并行加速比有理论上限。其公式为：

$$S = \frac{1}{(1 - f) + f/p} \quad (1)$$

其中， f 是可以被并行化的比例， p 是处理器数量， S 是加速比。例如，如果一个任务的80%可以并行化（ $f = 0.8$ ），使用4个处理器（ $p = 4$ ），则最大加速比为 $S = 1/(0.2 + 0.8/4) = 1/0.4 = 2.5$ 倍，而非理想中的4倍。即使处理器数量趋于无穷，加速比也趋近于 $1/(1 - f) = 5$ 倍。Amdahl定律的核心洞察是：串行部分决定了加速上限。Malekar和Zand于2024年将Amdahl定律适配到LLM场景，提出了面向LLM吞吐量的分析框架[?]

Roofline模型将一个算子的性能约束可视化[?]。对于给定硬件平台，一个算子的理论性能上界由两条线围成：水平线代表峰值计算能力（FLOP/s），斜线代表内存带宽乘以算术强度（Bandwidth $\times$ Arithmetic Intensity）。两条线的交点称为"ridge point"。当算子的算术强度低于ridge point时，它是内存带宽受限的（memory-bound）；高于ridge point时，它是计算受限的（compute-bound）。Yuan等人将Roofline模型系统应用于LLM推理分析，揭示了不同推理优化策略在带宽与计算之间的权衡[?]

局部性原理指导缓存设计：时间局部性（刚被访问的数据将再次被访问）和空间局部性（相邻地址的数据将很快被访问）[?]。这两个简单的经验法则催生了缓存行（cache line，通常64字节）、预取（prefetching）和缓存替换策略（如LRU）等一系列设计。

正是这些可量化的原则，使得计算机体系结构在八十年间实现了从每秒数千次浮点运算到每秒超过 10^{18} 次运算的性能增长，而程序员无需理解每一层内部的物理细节。

2.2 大模型系统的分层雏形

当前大模型系统正在经历与经典计算栈相似的分化过程。以下按照从底层到上层的顺序，逐一介绍每一层的发展现状。

2.2.1 模型层：推理核心的演进

在模型侧，Transformer[?]仍是主流骨架，其自注意力机制通过计算序列中任意两个位置之间的相关性来捕获长距离依赖。开放模型沿着几个方向持续演化：RoPE旋转位置编码[?]改进了位置信息的表达方式，使得模型能更好地处理长序列；MoE（Mixture of Experts）[?, ?]通过稀疏激活（每次推理只使用部分参数）来扩大模型容量而不成比例增加计算量；长上下文扩展[?, ?]将模型的有效上下文窗口从数千token扩展到数十万甚至数百万token。同时，Mamba[?]等选择性状态空间模型开辟了线性复杂度序列建模范式，不再依赖自注意力机制。

2.2.2 推理层：系统化的服务优化

在推理侧，一系列工作将模型推理从“单次前向传播”重构为系统化的工程问题。FlashAttention[?, ?]将注意力计算重新表述为“I/O感知”问题：通过分块计算（tiling）减少对HBM的访问次数，将注意力内存复杂度从 $O(N^2)$ 降低到 $O(N)$ 。vLLM用PagedAttention[?]将KV缓存管理表述为分页问题：把KV缓存划分为固定大小的“页”，按需分配和回收，避免了为每个请求预分配最大内存的浪费。ORCA[?]系统化地处理连续批调度（continuous batching），允许新请求在旧请求的解码间隙加入批次。DistServe[?]将“预填充”（prefill，即处理输入prompt）和“解码”（decode，即逐token生成）解耦到不同的GPU上，避免两者的资源竞争。Sarathi-Serve[?]进一步优化了吞吐与延迟之间的折中。这些工作的共同特点是将经典系统优化技术（分页、批调度、资源隔离）应用于LLM推理引擎。

2.2.3 记忆层：有限窗口与持久状态的协同

在记忆侧，核心挑战是：上下文窗口有限（如128K token），但Agent需要访问的知识和历史交互可能远超此限制。多条技术路线正在并行探索。长上下文扩展[?, ?]试图直接增大窗口容量，但面临计算成本随长度二次增长的问题。RAG（Retrieval-Augmented Generation）[?]在推理时从外部知识库检索相关片段注入上下文，相当于“按需加载”——类比操作系统中的demand paging。MemGPT[?]明确提出“虚拟上下文管理”，将上下文分为主上下文（working context，类比主存）和外部存储（external storage，类比磁盘），在两者之间自动移动信息。LongMem[?]和Generative Agents[?]探索了不同形式的长期记忆机制。LongMemEval[?]建立了长期交互记忆的评测基准。Letta[?]将状态化Agent作为核心设计，使Agent能够跨会话保持记忆。MemOS[?]更进一步，将记忆视为操作系统级的一等资源，统一管理文本记忆、激活记忆等不同表示形式。MemoryOS[?]在EMNLP 2025上提出了面向AI Agent的记忆操作系统。上下文工程（Context Engineering）[?]作为一个新兴学科正在系统化这一领域。它被定义为“系统性地设计、选择和优化输入给LLM的全部信息”的工程实践，涵盖检索增强、记忆管理、工具使用scaffolding和多轮对话优化。

2.2.4 Agent层：从单轮推理到复杂运行时

在Agent侧，大模型已不再只是“单轮推理器”，而是复杂运行时中的控制器。ReAct[?]将推理（Reasoning）与行动（Acting）交织，让模型在“思考-行动-观察”的循环中解决多步任务。Toolformer[?]让模型学会自主调用外部工具（如计算器、搜索引擎）。SWE-agent[?]和OpenHands[?]是面向软件工程的自主Agent，能够浏览代码仓库、编写补丁、运行测试。Codex[?]和Claude Code[?]是工业级的代码Agent，具备子Agent调度[?, ?]、沙箱隔离[?, ?]和权限控制[?, ?]等类似操作系统内核的能力。多Agent协作框架（如AutoGen[?]、MetaGPT[?]）进一步将多个专业化Agent组织成协作网络。

2.3 缺少统一的系统语言

问题在于，如果没有统一的系统语言，上述各层的研究很容易变成零散的技巧集合。例如，*prefix caching*（前缀缓存复用）、*chunked prefill*（分块预填充调度）、*subagents*（子Agent并

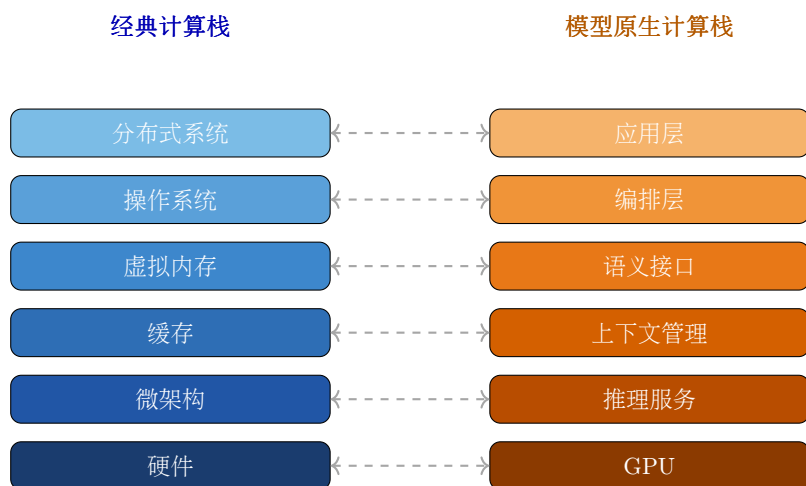


Figure 2: 经典计算栈与模型原生计算栈的分层对比

发执行）、*MCP servers*（标准化工具接口）、*persistent memory*（持久化记忆存储）——这些概念看起来分属不同方向，但在体系结构视角下，它们分别对应缓存复用、调度粒度、并发执行、I/O接口和分层存储。

这正是本文采用计算机体系结构类比的原因：它并不试图证明两者“本质相同”，而是把相互独立的工程点，重新放回一个更大而统一的设计空间，使得不同层之间的优化可以被组合、被预测、被系统化地设计。

图 ??给出了经典计算栈与模型原生计算栈的分层对比。

3 相关工作

本文的核心命题是：用计算机体系结构（computer architecture）的类比框架来构想未来模型原生计算系统的完整分层设计。这一命题并非在真空中产生——到2025年，已有大量工作从各自角度触及了“LLM系统需要操作系统式的抽象”、“Agent需要记忆管理”、“工具调用需要标准协议”等关键洞察。本节按功能主题梳理最相关的文献，逐步揭示每条线索的贡献与局限，最终定位本文的独特贡献。

3.1 LLM-as-OS / Agent Kernel: 将大语言模型视为操作系统内核

最早系统化地提出“LLM与操作系统类比”的研究，是Ge等人在2023年的工作“LLM as OS, Agents as Apps”[?]。这项工作的核心洞察可以用一张系统映射图来理解：LLM对应OS kernel（负责核心推理与调度），context window（上下文窗口）对应内存，外部存储对应文件系统，硬件工具对应外设，软件工具对应编程库，用户提示对应用户命令。换言之，他们将LLM不是一个“聊天机器人”，而是一个“内核进程”——这一比喻后来被Karpathy在社交媒体上进一步传播[?]。在本文的类比体系中，Ge等人的工作直接对应“第2层Agent OS”的概念萌芽。

Mei等人的AiOS[?]将这一思路从类比推进到可运行的系统实现。AiOS的核心设计是将Agent运行时（runtime）中原本与应用逻辑耦合的职责剥离出来，放入一个专门的“kernel”中，包括：(1) 调度器——管理多个Agent的并发执行；(2) 上下文管理——在多个Agent间分配有限的context window；(3) 记忆管理——维护短期与长期记忆的读写；(4) 存储管理——管理文件与持久化数据；(5) 访问控制——控制Agent对工具和数据的权限。实验报告显示，这种解耦设计带来了最高2.1倍的运行加速。在本文的类比体系中，AiOS实现了一个早期的Agent OS原型，其kernel职责划分直接对应传统OS中的进程管理、内存管理和文件系统模块。

AgentOS[?]进一步将LLM定义为受"structured operating system logic"约束的"reasoning kernel"——即LLM不是自由的推理引擎，而是必须在操作系统逻辑框架内运行的受控推理核心。这一视角强调的是LLM的推理能力需要被结构化的系统逻辑所约束和编排，而不是让模型无约束地自由生成。

到2025–2026年，这条研究线从学术原型向实际桌面和移动场景推进。微软的UFO²[?]将HostAgent（理解用户意图）、AppAgent（操作特定应用）、GUI-API混合动作层和虚拟桌面并行操作组织成"Desktop AgentOS"。其创新在于同时支持GUI截图理解与原生API调用两种交互方式。Aura[?]则面向移动场景，采用Hub-and-Spoke拓扑：一个特权System Agent解析用户意图，多个沙箱化的App Agent各自负责一个领域。其设计哲学是"安全优先"——Agent间的通信必须经过System Agent的中介，类似OS中进程间通信必须经过内核。

与本文的关系 上述工作各自定义了LLM在系统中的角色（OS kernel / reasoning kernel / Agent Kernel），但尚未形成统一的分层模型。本文在第??节提出的ICAM模型将这些不同的"kernel"概念统一到一个6层体系结构中，并明确LLM应被定位为第1层（概率计算核心），而Agent OS是第2层的独立抽象——两者不应混为一谈。

3.2 记忆层级与虚拟上下文管理

大语言模型的context window（上下文窗口）容量有限——例如GPT-4 Turbo为128K tokens，Claude 3为200K tokens，Gemini 1.5 Pro可达1M tokens[?]。然而，无论模型窗口多长，实际可用容量始终是有限的，这与传统计算机中"物理内存容量有限"的问题完全同构。因此，一系列工作借鉴操作系统中的**内存管理**技术来扩展Agent的可用上下文。

MemGPT[?]（后演化为Letta平台）是这一方向的奠基性工作。其核心思路不是简单地加一个RAG（检索增强生成）[?]来查找外部知识，而是借鉴传统操作系统中的**虚拟内存管理机制**：将有限的context window类比为"主存"（main memory），将外部数据库类比为"磁盘"（disk），通过一个类OS的内存管理器自动地在context window和外部存储之间"换入换出"（page in/page out）信息片段，使得Agent获得一种"看上去拥有无限上下文"的能力。MemGPT的控制流还显式使用了interrupts（中断）机制——当需要从外部存储检索信息时，触发一个中断将控制权交给memory manager。这一设计直接对应本文类比中的"虚拟内存"概念。

MemOS[?]将问题提升到更系统的层次。它将三种不同粒度的记忆统一为可管理的系统资源：(1) plaintext memory——以文本形式存储的外部知识；(2) activation-based memory——模型内部的隐层表示（类似CPU寄存器）；(3) parameter-level memory——模型参数中编码的知识（类似固件/ROM）。MemOS用"MemCube"作为封装单元，将内容与溯源信息（provenance）、版本管理（versioning）打包在一起，并试图在retrieval（检索）与parameter learning（参数学习）之间建立迁移桥梁。这一工作对应本文类比中"从KV cache到参数存储的多级存储层次"。

MemoryOS[?]直接将OS内存管理原则应用于Agent记忆系统。HiAgent[?]（发表于ACL 2025）则提出了"层次化工作记忆管理"——以子目标（subgoal）作为记忆分块单元，将长时任务的记忆组织成层次结构，类似于操作系统中的多级页表（multi-level page table）。A-MEM[?]提出了"能动性记忆"（agentic memory）的概念——记忆不是被动存储的记录，而是可以由Agent自主组织、重组和更新的动态资源。LongMemEval[?]进一步说明，长期记忆不是"存更多聊天记录"这么简单，而是涉及信息抽取、时序推理、知识更新与拒答能力的复杂系统工程。

与本文的关系 记忆子系统的工作已经高度系统化，但其讨论通常独立于Agent运行时（runtime）和I/O层，缺少全栈视角。更重要的是，现有工作尚未明确将KV cache（模型推理时的键值缓存）作为类比中"硬件缓存"的对应物——而本文认为KV cache正是连接"概率计算核心"与"语义记忆层级"的关键硬件抽象。本文在第??节中详细阐述了这一对应关系。

3.3 Agent框架与运行时

如果说前两个主题分别定义了"内核"和"内存",那么本节聚焦于"程序是如何在内核上运行的"——即Agent的执行框架与运行时环境。

基础能力：推理与行动的交织 ReAct[?]是Agent执行范式的基础性工作。在此之前,LLM的推理(reasoning,如Chain-of-Thought)和行动(acting,如调用工具)是分离的两个步骤。ReAct的核心贡献是证明:将推理和行动**交织**(interleave)在同一流程中——即"思考一步,行动一步,观察结果,再思考"——可以显著提高任务完成质量。这类类似于传统编程中"CPU执行一条指令,访问一次内存,检查一个条件"的fetch-decode-execute循环。

Toolformer[?]解决了另一个基础问题:让模型自己学会**何时**调用API。通过在训练数据中自动插入API调用示例,Toolformer使模型在生成过程中自主决定是否需要调用某个工具——这类类似于操作系统中的"系统调用"(system call)机制,由用户程序主动发起请求。

编程Agent的兴起 2024-2026年最引人注目的Agent进展集中在软件工程领域。OpenAI的Codex CLI[?]于2025年4月发布,是一个运行在终端中的Agent编程助手。它能够读取代码仓库、编辑文件、执行命令,并在沙箱(sandbox)环境中安全运行[?]。2026年初,Codex CLI引入了subagent(子Agent)功能[?]——主Agent可以生成专门的子Agent并行执行子任务,每个子Agent拥有独立的上下文窗口和沙箱。这直接对应了传统OS中"父进程创建子进程"的fork模式。Codex还支持通过AGENTS.md文件定义自定义指令[?]和通过Skills定义可复用能力模块[?]。

Anthropic的Claude Code[?]是另一个代表性的Agent运行时。与Codex类似,它是一个终端原生的编程Agent,能够编辑文件、执行shell命令、连接外部服务。Claude Code的架构设计被学术研究系统地分析过[?]——研究发现其系统架构中仅约1.6%是AI决策逻辑,其余98.4%是运行时脚手架(operational harness),包括安全控制、工具编排、上下文管理等。Claude Code也支持subagent[?]和Skills[?]扩展,并通过CLAUDE.md文件实现项目级记忆[?]。

值得注意的是,Agent执行的时间跨度正在从分钟级向小时甚至天级扩展。Rakuten的工程师测试Claude Code在vLLM项目(1250万行代码)上完成激活向量提取任务,Agent自主运行7小时并达到99.9%数值精度[?]。这一趋势意味着Agent运行时必须处理远超当前交互式会话的状态持久化、检查点恢复和跨步骤资源管理问题。

OpenHands[?](前身OpenDevin)是一个开源的通用Agent平台,发表于ICLR 2025。它在SWE-bench Verified基准上达到约70-74%的问题解决率[?],支持多种LLM后端。SWE-agent[?]则专注于软件工程任务,用更简洁的Agent-Computer Interface(ACI)设计实现了相近的性能。

与本文的关系 这些Agent框架在架构上已经浮现出操作系统的特征: Codex的subagent对应进程管理, Claude Code的CLAUDE.md对应配置/环境管理, OpenHands的ACI对应系统调用接口。但它们目前仍是各自独立设计的"应用层"工具,缺乏统一的分层抽象。本文的ICAM模型为这些框架提供了统一的理论框架,将它们的功能映射到明确的体系结构层次中。

3.4 工具调用与I/O协议

Agent需要与外部世界交互——查询数据库、调用API、操作文件系统。这一需求在传统计算机中对应I/O子系统(输入/输出设备与总线)。近两年的关键进展是这种I/O抽象正在从各自为政的工具调用走向**标准化协议**。

HuggingGPT[?]是早期的代表性工作:将LLM放到"controller"(控制器)位置,由它解析用户任务、选择合适的专家模型、协调执行并汇总结果。这类类似于操作系统中的设备驱动管理层——由OS统一管理不同设备(模型)的调用。Gorilla[?]聚焦于一个更具体的问题:当API文档频繁更新时,模型如何保持调用的准确性?这对应了设备驱动版本兼容性的问题。

MCP: 模型到系统的I/O总线 2024年11月,Anthropic推出了Model Context Proto-

col (MCP, 模型上下文协议) [?], 定义为连接AI应用与外部数据源、工具、工作流的开放标准。MCP采用客户端-服务器架构: MCP Client运行在AI应用侧, MCP Server封装具体的工具或数据源。通过统一的JSON-RPC协议, 任何支持MCP的AI应用都可以连接任何提供MCP Server的工具——Anthropic官方将其比喻为"AI的USB-C"。到2026年, MCP生态已快速扩展, Claude Code[?]和众多第三方工具均支持MCP连接。

A2A: Agent到Agent的互联协议 2025年4月, Google在Cloud Next '25大会上推出Agent-to-Agent Protocol (A2A) [?], 明确定位为与MCP互补的开放协议。A2A解决的不是"模型到工具"的连接问题, 而是"Agent到Agent"的协作问题——支持能力发现 (capability discovery)、任务生命周期管理 (task lifecycle)、长时任务和多模态协同。A2A已于2025年捐赠给Linux基金会, 成为行业标准候选。A2A采用对等的客户端-服务器模型, 每个Agent既可以是发起请求的客户端, 也可以是响应请求的服务器。

与本文的关系 将两者合起来看, MCP扮演模型到系统的I/O总线角色 (类似USB/PCIe), A2A扮演Agent到Agent的互联协议角色 (类似TCP/IP网络协议栈)。在本文的ICAM模型中, 这分别对应第4层 (I/O与工具层) 和第5层 (互联与协议层)。现有工作尚未将这两个协议纳入统一的分层抽象中, 也没有在体系结构层面讨论其与上下文管理、内核调度等层的交互。

3.5 多Agent协作与分布式视角

当多个Agent协同工作时, 系统呈现出分布式计算的特征——这与从单机到分布式系统的演进完全平行。

AutoGen[?] (微软研究院, 2023年) 是多Agent系统的奠基性框架。其核心设计是将"多Agent会话" (multi-agent conversation) 作为通用基础设施——不同角色的Agent通过自然语言对话来协调任务。AutoGen已演进到v0.4版本, 并被整合进Microsoft Agent Framework中, 支持可扩展的生产级多Agent系统。

MetaGPT[?] (ICLR 2024 Oral) 引入了一个关键的设计思想: 将人类组织中的**标准操作流程** (SOP, Standard Operating Procedure) 显式编码到多Agent协作中。具体而言, MetaGPT模拟了一个虚拟软件公司: 产品经理写需求, 架构师设计系统, 工程师写代码, QA工程师测试——每个角色是一个Agent, SOP定义了它们之间的工作流程和传递方式。这类似于分布式系统中的工作流编排 (workflow orchestration), 将无序的Agent交互约束为有序的流水线 (assembly line)。

这一流水线模式已在工业规模上得到验证。Fountain的Copilot系统采用中心编排Agent协调候选人筛选、文档生成和情感分析三个专业子Agent, 实现了50%更快的候选人筛选速度、40%更快的入职流程和2倍的候选人转化率, 将招聘中心的全面配置周期从一周以上压缩到72小时以内[?]。这种"中心辐射" (hub-and-spoke) 拓扑与MetaGPT的流水线拓扑、AutoGen的对等拓扑形成了鲜明对比, 表明协调拓扑本身是影响多Agent系统性能的关键设计变量[?, ?]。

Internet of Agents (IoA) [?] (发表于ICLR 2025) 从"互联网"而非单机系统的视角来想象Agent生态。其核心贡献是提出了一种Agent集成协议 (agent integration protocol), 支持异构Agent之间的动态组队、协作和分布式执行。IoA的设计灵感来自互联网——正如互联网连接了不同厂商、不同操作系统的计算机, IoA要连接不同框架、不同公司开发的Agent。这也催生了IETF层面的标准化努力 (IoA Protocol Internet Draft)。

与本文的关系 多Agent系统已经在研究对象层面浮现出"分布式系统"的形态: AutoGen对应消息传递机制, MetaGPT对应工作流编排, IoA对应网络互联协议。但大多数工作仍停留在"框架和协议"层, 未与单体Agent的内存管理、内核调度、I/O、权限控制一起纳入统一的体系结构。本文的ICAM模型将多Agent协作映射到第5层 (互联与协议层) 和第6层 (应用层), 并讨论了跨层交互的设计原则。

3.6 认知架构与安全治理

前面各节从系统功能角度（内核、内存、运行时、I/O、多Agent）梳理了相关工作。本节转向两个更高层的主题：Agent系统应遵循什么样的认知架构？如何治理其安全性？

认知架构 CoALA (Cognitive Architectures for Language Agents) [?]从认知科学角度提供了一个理论框架。CoALA将语言Agent组织为三个核心组件：(1) 模块化记忆 (modular memory) ——分为短期工作记忆和长期记忆（语义记忆、情景记忆、程序记忆）；(2) 结构化动作空间 (structured action space) ——内动作（推理、记忆检索）与外动作（工具调用、环境交互）；(3) 通用决策过程 (generalized decision-making process) ——规划、执行、观察的循环。CoALA的价值在于提供了“为什么这个系统不是纯软件栈拼装，而是真正的认知架构”的理论支点。L2MAC[?] (ICLR 2024) 则明确将其框架称为“first practical LLM-based stored-program automatic computer (von Neumann architecture)”，包含指令寄存器、文件存储、控制单元和独立的LLM Agent模块。Mi等人[?]从“computer systems perspective”组织LLM Agent综述，明确表示受von Neumann architecture启发，把Agent拆成感知、认知、记忆、工具、行动等模块，并指出context window类似主存、数据库类似磁盘，而Agent目前缺少的正是“cache module”。

安全治理 ArbiterOS[?]指出了一个根本性问题：我们正在用**确定性软件工程** (deterministic software engineering) 的思维模型来指挥**本质上概率性的处理器** (inherently probabilistic processors)。这一矛盾要求在系统架构层面引入治理层。ArbiterOS由此提出“Agentic Computer”心智模型——将LLM重新定义为“Probabilistic CPU” (概率CPU)，并设计governor (治理器)、formal instruction set (形式化指令集) 和HAL (硬件抽象层) 等治理层概念。在本文的类比体系中，ArbiterOS的核心洞察——模型是概率性的、需要确定性治理层——正是本文提出“双平面架构” (概率计算平面+ 确定性控制平面) 的直接灵感来源之一。

CaMeL[?] (Google DeepMind, 2025) 从安全角度将操作系统能力安全原则应用于LLM Agent。其核心设计理念是“Don't execute data”——严格分离可信指令与不可信数据，通过提取控制流和数据流并分别施加安全策略来防御提示注入攻击。IronClaw[?] (2026) 明确提出要像操作系统一样保护AI Agent——将OS中的权限分离、沙箱隔离、审计日志等安全机制迁移到Agent系统中。

与本文的关系 认知架构线提供了理论根基，安全治理线提供了工程约束。但这两条线之间缺乏统一——认知架构通常不讨论安全性，安全治理通常不讨论认知结构。本文的双平面架构试图统一两者：概率计算平面承载认知能力，确定性控制平面承载安全治理，两者通过明确定义的接口交互。

3.7 现有工作的共同边界与未解决问题

上述六个主题共同逼近了本文设想的“模型原生计算体系结构”，但存在四个尚未被统一解决的关键问题。

图 ??给出了现有工作对各系统层次的覆盖情况。

问题一：LLM到底扮演什么角色？ 现有工作对LLM在系统中的定位存在根本分歧。Ge等人将LLM放到OS核心位置[?, ?]——LLM就是kernel本身；ArbiterOS将其下沉为被kernel治理的“Probabilistic CPU”[?]——LLM是被管理的处理器；AgentOS将其定义为受structured OS logic约束的reasoning kernel[?]——LLM是受约束的推理核心。这三种定位导致了完全不同的系统设计。本文提出的ICAM模型给出统一答案：LLM是第1层（概率计算核心/CPU），Agent OS是第2层（操作系统内核），两者是不同的抽象层次，不应混为一谈。LLM提供计算能力，Agent OS提供管理与调度——正如CPU与OS kernel在传统体系结构中是两个独立但紧密协作的层次。

问题二：缺乏全栈分层模型 没有哪篇主流工作同时将概率计算核心、语义记忆层级、工具与外设总线、Agent OS/kernel、互联协议、治理安全层、编程模型/指令语义、评价

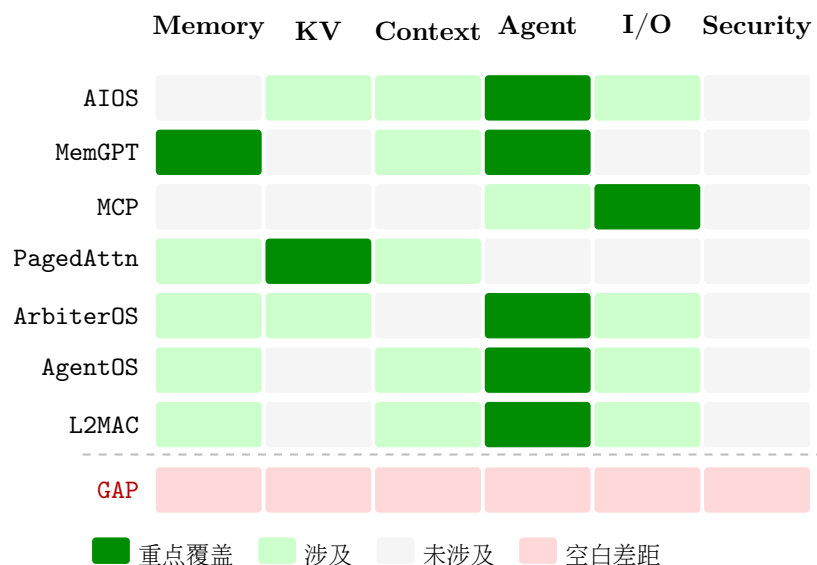


Figure 3: 现有工作对各系统层次的覆盖情况

指标全部放在同一套体系结构中。L2MAC[?]聚焦stored-program计算机模型，Mi等人[?]提供了von Neumann启发的综述框架，但各自只覆盖了体系结构的一个侧面。记忆管理的工作不讨论I/O协议，Agent框架不讨论KV cache，安全治理不讨论多Agent协作——每条线的贡献都是真实的，但边界也是清晰的。

问题三：概率性与确定性的矛盾未被架构化 ArbiterOS[?]正确地指出了概率性处理器与确定性治理之间的矛盾，但只是提出了概念层面的"governance-first"范式。CaMeL[?]和IronClaw[?]提供了具体的安全机制，但未将其嵌入完整的分层架构中。本文通过"双平面架构"将这一矛盾架构化：概率计算平面承载LLM的推理能力，确定性控制平面承载OS的管理与安全职责，两个平面通过明确的接口协议协作。

正是这三个未解决的问题，定义了本文的贡献空间：提供一个统一的、分层的、双平面的体系结构模型，将上述六条线索整合在同一张总图中。

问题四：人机协作的结构约束未被建模 现有工作将人视为系统的边界条件（L6用户），但最新的工业实证揭示了一个"协作悖论"：工程师在工作中约60%使用AI，却只能完全委托0-20%的任务[?]。工程师倾向于委托"易于验证或风险较低"的任务，而保留"概念上困难或依赖设计决策"的任务。这一结构性约束暗示人机交互环路本身应成为体系结构的显式组成部分，而非外部假设。

4 类比框架：把智能系统看作一台分层计算机

4.1 类比的基本原则

本文采用的类比框架见表 ??。在详细展开之前，有必要先阐明三条基本原则，以避免类比沦为牵强的修辞。

原则一：类比必须对应真实的工程机制 “KV缓存像缓存”之所以成立，不是因为名字相似，而是因为二者都服务于“时间局部性”（temporal locality）和“状态复用”的核心目标[?, ?]。所谓时间局部性，是指“最近被访问过的数据，很可能在不久的将来再次被访问”——无论是一块CPU缓存行，还是一组刚刚被计算的KV向量，都遵循同一规律。“PagedAttention像虚拟内存”之所以成立，是因为vLLM确实把KV块组织成固定大小的页，通过block table映射到非连续物理空间，与操作系统管理虚拟页的机制在工程实现上高度一致[?]

原则二：类比必须承认边界 模型不是确定性CPU——CPU执行ADD R1, R2每次结果完

全相同，而模型对同一提示词可能产生不同回复；语义检索不是精确寻址——`MOV [0x7fff], EAX`总是写入同一地址，而向量相似度搜索返回的结果随嵌入模型和查询措辞而变化；Agent运行时也不是成熟OS——Linux内核有40年的形式化验证和回归测试，而当前Agent框架的安全机制仍在快速迭代中。这些“不完全相同”恰恰构成了未来研究最值得投入的空间。

原则三：类比的目标是生成研究问题 一个好的类比不仅帮助理解，更重要的是提示新的工程问题：哪些模块之间需要定义清晰的接口？哪些共享状态需要一致性保证？哪些执行路径需要权限控制和审计日志？表 ??最后一列“工程意义”正是类比所生成的研究方向。

在这个框架下，未来模型原生计算架构可被理解为：“以基础模型为计算核心，以KV和上下文层次为存储层，以Agent内核为控制层，以工具和外部世界为I/O层，以服务集群和共享记忆为分布式底座”。接下来，我们将按照从核心到外设的顺序，逐一展开每个组件的类比分析。

4.2 隐喻冲突与双平面架构

在深入组件之前，需要先解决一个根本性的概念冲突。如第 ??节所述，现有文献对“LLM在体系结构中占据哪一层”存在根本分歧：Ge等人把LLM放在OS kernel的位置[?]; AIOS将其与资源管理、上下文管理、访问控制一起纳入kernel式服务[?]; AgentOS把LLM视为由structured operating system logic约束的“reasoning kernel”[?]; ArbiterOS又把LLM下沉为受governor治理的“Probabilistic CPU”[?]

这些定位看似矛盾，但实际上各自只看到了系统的一个侧面。本文认为，模型原生计算系统实际上同时包含**两个平面**，不同工作分别只看到了其中一个。

4.2.1 概率执行平面 (Probabilistic Execution Plane)

什么是概率执行平面？ 简单来说，就是“大模型本身”——基础模型的前向推理过程。

具体例子：当用户向GPT-4输入一段代码审查请求时，模型的注意力机制会在数十亿参数上进行矩阵运算，最终输出一个token概率分布，从中采样出回复。这个过程中，同样的输入可能产生不同的输出，输出的质量高度依赖于上下文的组织方式，且模型本身不持有任何持久状态——前一次对话的结果不会自动保留到下一次。

这一平面的特征：

- 概率式输出：输出服从概率分布，同一输入可产生不同结果
- 上下文高度敏感：提示词的微小变化可能显著改变输出质量
- 无持久状态：模型权重在请求之间不变，所有“记忆”都来自上下文窗口中的token

KV缓存、注意力机制、解码策略都属于这一平面的“微架构”——它们优化的是模型前向计算本身的效率和精度。

4.2.2 确定性控制平面 (Deterministic Control Plane)

什么是确定性控制平面？ 简单来说，就是“围绕模型的程序化控制层”——调度器、权限审批器、状态机、日志系统、失败恢复逻辑。

具体例子：当Claude Code收到“帮我重构这个函数”的请求时，确定性控制平面执行以下操作：检查当前目录权限（是否在允许的项目路径内）→ 加载项目记忆文件（`CLAUDE.md`）→ 解析请求并分解为子任务→ 对每个涉及文件写入的步骤，根据审批策略决定是否需要用户确认→ 执行完毕后将变更日志写入审计记录[?]。这些步骤全部由确定性代码执行，行为可预测、可审计、可回放。

这一平面的特征：

- 确定性执行：相同的输入和策略配置总是产生相同的控制决策
- 形式化策略：权限规则、审批策略以代码或配置文件明确定义



Figure 4: 双平面架构：概率执行平面（蓝，L1–L3）与确定性控制平面（橙，L4–L5）的分离

- 可审计：每个决策步骤都有日志记录，支持事后追溯

Codex的approval policies[?]、Claude Code的只读默认与hooks[?]、CaMeL的capability安全机制[?]都属于这一平面。

4.2.3 为什么需要分开？

工业界的实践已经验证了这一分离的必要性。Praetorian的开发平台将LLM嵌入确定性编排层，显著提升了Agent在自主软件开发中的可靠性[?]。“确定性外壳、概率核心”（Deterministic Shell, Probabilistic Core）模式正成为Agent系统的事实标准架构[?]。“Blueprint First, Model Second”框架将运营逻辑编码为确定性源代码蓝图，约束LLM在明确定义的边界内行动[?]。

图 ??展示了双平面架构中概率执行平面与确定性控制平面的分离。

分离的核心理由是职责不同：

- 概率平面擅长理解和生成——它“能做什么”
- 确定性平面擅长控制和保障——它“应该做什么”
- 概率平面不应直接执行有副作用的操作（如写入文件、调用外部API），而应通过确定性平面的审批和日志通道间接执行

这种分离得到了工业实践的有力验证。Anthropic的内部研究表明，工程师在工作中约60%使用AI，但只能“完全委托”0–20%的任务[?]。这一“协作悖论”并非模型能力的暂时不足，而是双平面系统的结构性特征：确定性控制平面（包括通过权限策略和审批流程传递的人类判断）自然地约束了概率平面的行动空间。工程师倾向于委托“易于验证或低风险”的任务而保留“概念上困难或依赖设计决策”的任务，这恰恰是确定性平面作为概率平面“门控器”的行为表现。如果两个平面没有分离——即模型可以直接执行任何操作——则这种渐进式委托将不可能实现。

这与传统计算机中“用户态不能直接访问硬件，必须通过系统调用陷入内核”的设计哲学完全一致[?]。用户程序（概率平面）可以发出任意请求，但内核（确定性平面）决定哪些请求被允许、以什么优先级执行、如何记录和恢复。

冲突因此消解：当ArbiterOS把LLM称为“Probabilistic CPU”时，它看到的是概率执行

平面；当AIOS把LLM纳入“kernel”时，它看到的是概率执行平面与部分控制逻辑的混合体；当AgentOS提出“reasoning kernel”时，它在尝试用控制平面的结构来约束概率平面。这些工作并不矛盾，而是从不同角度描述了同一个双平面系统。

5 关键组件详析

本节按照从核心到外设的顺序，依次分析六个关键组件：基础模型（对应CPU）→ KV缓存（对应缓存层次）→ 上下文管理（对应虚拟内存）→ Agent运行时（对应操作系统）→ 工具总线（对应I/O）→ 多Agent协作（对应分布式系统）。对于每个组件，我们都遵循四步结构：(1) 经典计算机中的对应概念是什么；(2) 大模型系统中的对应物是什么；(3) 两者为什么相似（机制层面的解释）；(4) 本质差异是什么（不回避问题）。

5.1 基础模型：概率式模型计算核

5.1.1 经典计算机中的CPU

在经典计算机中，CPU（中央处理器）是整个系统的执行核心。它的职责可以用一句话概括：**按指令序列执行操作，将输入状态映射为输出状态**。CPU内部包含算术逻辑单元（ALU）用于计算、寄存器文件用于暂存操作数、控制单元用于取指译码。关键特征是**确定性**：执行ADD R1, R2, R3一百万次，只要输入寄存器的值相同，结果就一定相同。CPU的行为由指令集架构（ISA）精确定义，例如RISC-V规范对每条指令的行为给出了数学级的描述[?]。

5.1.2 大模型系统中的对应物

在大模型系统中，基础模型（如GPT-4、Claude、Llama）承担了类似CPU的“通用执行核心”角色。给定输入上下文（包含系统提示、用户消息、工具结果与控制约束），模型通过前向计算产生下一步动作或输出[?, ?, ?]。

提示词、工具描述、Skill文件、项目记忆文件（如AGENTS.md或CLAUDE.md）可以被理解为一种**软指令接口**：它们不改变模型的物理权重（就像软件不改变CPU的电路），却显著改变模型的执行路径[?, ?, ?]。例如，一个精心编写的CLAUDE.md文件可以让同一个模型在Python项目中表现出“Python专家”的行为模式，在Rust项目中表现出“Rust专家”的行为模式——就像同一个CPU加载不同的程序就表现出不同的行为。

SGLang已经朝这个方向迈进：它把复杂的语言模型程序重新表述为结构化前端语言与高性能运行时的协同，类似于从汇编语言向高级语言的演化[?, ?]。

5.1.3 为什么相似

CPU和基础模型之间的相似性体现在三个层面：

第一，通用性 CPU是通用计算核心——同一个CPU可以运行文字处理器，也可以运行科学模拟程序。基础模型也是通用推理核心——同一个模型可以写代码、分析数据、翻译语言、规划任务。这种通用性使得两者都可以作为“硬件基础”，上层通过不同的“软件”（程序vs. 提示词）来定制行为。

第二，接口抽象 CPU通过ISA向上层暴露操作接口（算术、跳转、访存）；模型通过提示模板和工具schema向上层暴露操作接口（推理、生成、工具调用）。两者都遵循“接口稳定、实现可演进”的原则——x86架构存在了40多年，而底层微架构从单标量演进到了超标量乱序执行；GPT-4的API接口保持稳定，而底层模型版本持续更新。

第三，微架构优化空间 CPU有流水线、分支预测、超标量等微架构优化；模型有FlashAttention（IO感知的注意力计算[?]）、推测解码（用小模型预测大模型输出[?]）、连续批处理等微架构优化。两者都面临“接口不变、底层性能持续提升”的工程挑战。

5.1.4 本质差异

差异的核心在于**确定性vs. 概率性**。CPU执行形式化指令集，每条指令的语义精确定义、长期稳定、完全可复现。模型处理的是自然语言、代码和结构化schema的混合输入，输出服从概率分布。正如Mi等人所指出的，从计算机体系结构的演进中汲取灵感来设计Agent系统，需要同时承认概率式执行与确定性执行的根本差异[?]。

因此，对未来“智能ISA”的理解不应是把提示词硬编码为固定指令，而应是发展更稳定的接口层：工具schema、受控输出格式、可复用技能包、任务DSL以及对上下文的显式声明。从体系结构观点看，这意味着未来研究不应只比较模型参数量，还应比较“模型可编程性”——即上层能在多大程度上稳定、可靠地控制模型行为。

5.2 KV缓存：智能系统的缓存层次

5.2.1 经典计算机中的缓存层次

在经典计算机中，缓存是解决“处理器快、内存慢”这一根本矛盾的关键机制[?]。现代CPU通常有三层缓存：L1（约1ns、64KB）→ L2（约4ns、512KB）→ L3（约12ns、数MB到数十MB）。缓存利用**时间局部性**（temporal locality）：最近被访问的数据很可能再次被访问。当CPU需要数据时，先查L1，未命中则查L2，再未命中则查L3，最终从主存加载。缓存的管理涉及三个核心问题：放置策略（数据存在哪里）、替换策略（满了淘汰谁）、写策略（修改如何同步）。

5.2.2 大模型系统中的对应物

在自回归推理中，生成每个新token时都需要用该token的Query与之前所有tokens的Key/Value计算注意力。如果每生成一个token都重算整个前缀的KV，计算量随序列长度呈 $O(n^2)$ 增长。KV缓存通过保存已计算的历史tokens的Key和Value向量，将每步注意力计算简化为 $O(1)$ 的查表操作，使整体推理从 $O(n^2)$ 降为 $O(n)$ 。这与经典缓存的目的高度一致：**牺牲容量（显存）换取延迟与带宽优势**[?, ?]。

KV缓存的层次结构正在形成：

- **L1——会话级KV缓存：**单次会话中的KV向量，延迟最低，容量受GPU显存限制
- **L2——前缀/共享缓存：**跨请求共享的系统提示或代码库索引的KV块，类似于CPU中多进程共享的只读代码页[?, ?, ?]
- **L3——语义缓存：**相同问题类下可复用的工具调用结果、相同仓库结构下可复用的索引，属于“语义级别”的缓存命中

当vLLM引入PagedAttention，把KV组织成固定大小block并通过block table映射到非连续物理空间时，这种类比已经不只是启发式，而是直接借用了虚拟内存的页式管理思想[?]。

5.2.3 为什么相似

KV缓存与CPU缓存的相似性体现在工程机制的层面：

第一，都利用时间局部性 CPU缓存保存最近访问的缓存行，因为“最近用过的数据很快会再用”；KV缓存保存最近计算的KV向量，因为“生成下一个token时必须attends to 所有之前的token”。两者都把“需要反复使用的数据”保存在高速存储中，避免昂贵的重复计算或访存。

第二，都面临容量压力与替换决策 L1缓存容量有限，需要LRU等替换策略决定淘汰哪些缓存行；KV缓存同样面临显存压力，H2O通过识别“重击者”token（attention权重最大的token）实现了智能驱逐策略[?]，类似于CPU缓存中识别“热数据”的优化。StreamingLLM发现初始token获得的注意力不成比例地高（“attention sinks”现象），通过保留这些sink token实现了任意长度的流式推理[?]——这类似于CPU缓存中的“钉住”（pinning）策略。

第三，都存在带宽瓶颈 Gholami等人指出，LLM推理在解码阶段是典型的内存带宽受限问题，而非计算受限[?]。Yuan等人用Roofline模型系统分析了这一现象[?]。Li等人2024年的综述将KV缓存管理策略系统分为token级（决定哪些KV条目参与注意力）、模型级（架构优化如GQA/MQA）和系统级（内存管理与调度）三个层次[?]——这恰好对应了CPU缓存优化的多层次方法。

第四，压缩是共同的优化方向 KVQuant实现了8倍压缩[?]; KIVI通过非对称2-bit量化在几乎无损的条件下大幅减少显存占用[?]。这类似于CPU缓存中通过数据压缩（如IBM MXT）增加有效容量的思路。

5.2.4 本质差异

第一，命中判定标准不同 CPU缓存命中是布尔判断——地址相等则命中，否则不命中。KV缓存中，前缀缓存的确是精确匹配（前缀token序列相同则复用），但上层的语义缓存命中是**连续值比较**——语义相似度超过阈值即视为命中，不同阈值导致不同的精确率/召回率权衡。这意味着传统缓存理论中的命中率模型不能直接适用于语义缓存。

第二，失效机制不同 CPU缓存失效由写入一致性协议（如MESI）保证——数据被修改时，相关缓存行被标记为失效。KV缓存和语义缓存的“失效”远为复杂：工具版本变化、代码仓库提交变化、索引过期都会导致缓存命中但答案失真——即“语义失效”。近期研究已开始探索学习式的驱逐策略[?]和依赖感知的失效机制[?]来应对这一挑战。

第三，容量-延迟权衡更为严峻 CPU缓存层次中，L1→L2→L3→主存的延迟以纳秒级递增；在KV缓存中，从GPU显存→CPU内存→磁盘/网络的延迟跳变达到毫秒级，中间没有平滑过渡。未来缓存管理需要把传统cache policy与provenance、TTL、哈希版本一起设计。

5.3 上下文管理：语义版虚拟内存

5.3.1 经典计算机中的虚拟内存

在经典计算机中，虚拟内存解决了一个核心矛盾：“程序需要的地址空间远大于物理内存容量”。操作系统通过**分页机制**制造了“大空间”的幻觉：每个进程拥有独立的虚拟地址空间（如48位地址，256TB），但物理内存可能只有16GB。当进程访问一个尚未映射到物理内存的虚拟页时，触发“页缺失”（page fault），操作系统从磁盘加载所需页面。分页的核心优势是**按需加载**——只有实际被访问的数据才占用物理内存，且这个过程对应用程序完全透明。

虚拟内存管理的关键组件包括：页表（虚拟地址到物理地址的映射）、TLB（加速地址转换的缓存）、换页策略（决定哪些页被换出到磁盘）和写回策略（脏页何时写回磁盘）[?]。

5.3.2 大模型系统中的对应物

上下文窗口之于大模型，正如主存之于进程：它是高带宽、低延迟、但容量有限的工作区。Anthropic将context window直接定义为模型生成时可回看的working memory。

当前，上下文窗口容量正在快速扩展：Gemini 1.5 Pro支持高达1000万token的多模态上下文[?]; Ring Attention通过在多设备间环形分布序列实现了近乎无限的上下文长度[?]; RoPE/YaRN/LongRoPE等位置编码方法旨在增加可寻址历史范围[?, ?, ?]。

但大量研究表明，“声明可支持的窗口长度”与“真正有效的工作集容量”不是一回事。RULER、LongBench v2和LOFT对长上下文能力给出了更现实的压力测试[?, ?, ?]。就像一台宣称有256TB虚拟地址空间的计算机，实际可用内存可能只有16GB，真正的性能取决于工作集（working set）是否常驻物理内存。

因此，未来更合理的设计不应是“把所有东西一次性塞进上下文”，而应像虚拟内存那样做**分层管理**。

正在形成的分层方案：

- **热记忆（上下文窗口内）**：当前任务的完整信息，类似物理内存中的活跃页
- **温记忆（摘要与索引）**：经压缩的历史上下文，类似换出但保留摘要信息的页
- **冷记忆（外部知识库）**：持久化的向量库、文档库，类似磁盘上的数据

MemGPT把这种思想称为**虚拟上下文管理**，明确提出LLM作为操作系统的类比，将信息在上下文窗口（“主存”）和外部存储（“磁盘”）之间分页调度[?]。LongMem则把长期记忆显式地从主干模型中解耦出来[?]。RAG在这里扮演的更像**外存页入机制**——当Agent需要“页缺失”的信息时，通过语义检索从知识库中加载相关内容到上下文窗口[?]。MemoryOS更进一步，直接将OS内存管理原则应用于Agent记忆系统[?]。MemOS提出了完整的记忆操作系统架构[?]。HiAgent受人类问题解决过程启发，用子目标作为记忆块来层次化管理Agent工作记忆[?]。A-MEM提出让Agent自主组织记忆结构，无需预定义schema[?]。

5.3.3 为什么相似

上下文管理与虚拟内存的相似性体现在**机制层面**：

第一，都制造“大空间”幻觉 虚拟内存让每个进程以为自己拥有连续的大地址空间，实际物理内存远小于此。虚拟上下文管理让Agent以为自己拥有无限记忆，实际上下文窗口只有128K-1M token。两者都通过**按需加载**实现幻觉：只有当前需要的信息才占据“物理内存”（上下文窗口），其余信息存储在“磁盘”（外部知识库）中。

第二，都面临“地址转换”问题 虚拟内存通过页表将虚拟地址映射到物理地址；虚拟上下文通过语义检索将“信息需求”映射到具体的知识条目。只不过前者是精确匹配（页号→物理页号），后者是近似匹配（查询意图→Top-K相关文档）。

第三，都需要“换页策略” 当物理内存不足时，OS必须决定哪些页被换出（LRU、Clock等策略）；当上下文窗口快满时，Agent必须决定哪些信息被移出（摘要压缩、选择性遗忘）。这正是“上下文编译器”的核心功能——系统必须决定哪些状态以原文加载，哪些以摘要加载，哪些只保留索引，哪些应被丢弃[?]。

5.3.4 本质差异

关键差异：虚拟内存是无损的，上下文管理是有损的 OS的分页机制对应用程序完全透明——页换入换出不会改变任何一个字节。而Agent的上下文管理涉及**语义压缩**——摘要必然丢失细节，选择必然遗漏信息。“地址转换”也不是精确的：向量相似度搜索返回的是“最相关的”结果，而非“完全匹配的”结果。

这意味着语义虚拟内存永远不是无损页式系统。它更接近“带语义摘要和证据回填的分页内存”——信息被压缩后可能失真，但可以通过溯源（provenance）链回到原始数据验证。JetBrains的研究指出，高效上下文管理的关键不在于“塞入更多token”，而在于“减少噪声、保留信号”[?]——这进一步验证了有损压缩的必要性。

5.4 Agent运行时：智能操作系统

5.4.1 经典计算机中的操作系统

在经典计算机中，操作系统是“管理硬件资源、隔离应用程序、提供统一服务接口”的系统软件[?]。它的核心职责可以用三个抽象概括：**虚拟化**（让每个进程以为独占CPU和内存）、**并发**（让多个进程安全地同时运行）、**持久化**（可靠地存储和管理文件）。

具体来说，OS要做的事情包括：响应系统调用（如文件读写）、调度进程（哪个先运行、运行多久）、隔离进程（一个崩溃不影响另一个）、管理内存（分配、回收、换页）、控制权限（谁能访问什么资源）、处理I/O（与外部设备交互）、记录日志（审计与故障恢复）。

5.4.2 大模型系统中的对应物

如果模型是CPU，那么Agent框架越来越像OS。Agent运行时要做的事情包括：

- **解析目标**：将用户请求分解为可执行的子任务（类似程序加载与解析）
- **决定是否规划**：简单请求直接执行，复杂请求需要多步规划（类似短进程快速路径vs. 长进程调度）
- **装入上下文**：加载系统提示、项目记忆、历史对话到上下文窗口（类似进程的内存映像加载）
- **执行工具调用**：通过MCP等协议调用外部工具（类似系统调用）
- **审批危险动作**：对有副作用的操作（文件写入、代码执行）进行权限检查（类似capability-based访问控制）
- **并发启动子Agent**：将子任务分派给独立的子Agent执行（类似fork新进程）
- **整合中间结果**：汇总子Agent的输出，形成最终答案（类似进程间通信与结果合并）
- **状态持久化**：将重要状态写入记忆文件或知识库（类似文件系统写入）
- **失败恢复**：出错时重试或回滚，保证任务一致性（类似事务恢复）

这一类比的学术化表达已经出现。Ge等人提出“LLM as OS, Agents as Apps”的愿景[?]。Mei等人提出AIOS，构建了完整的LLM Agent操作系统内核架构[?]。Mi等人系统地计算机体系结构的演进经验应用于Agent系统设计[?]。Karpathy在2023年指出“LLM不是聊天机器人，而是新操作系统的内核进程”[?]——文件系统变成向量数据库，浏览器变成互联网搜索。这一愿景正在成为经济现实：一家Augment Code的企业客户将其CTO预估需要4-8个月的项目在2周内完成[?]，证明了有效的L3上下文管理与L5编排相结合可以将开发者入职和项目交付周期从周级别压缩到天级别。

工业实践也在朝这个方向演化：Codex提供subagents、skills、sandbox等控制面[?]；Claude Code提供hooks、MCP、memory文件与只读审批[?]。

5.4.3 为什么相似

Agent运行时与操作系统的相似性是所有类比中最为深刻的，因为它体现在**架构职责**的层面：

第一，都是资源管理者 OS管理CPU时间片、内存页、磁盘空间、网络带宽；Agent运行时管理模型推理配额、上下文窗口空间、工具调用频率、子Agent数量。两者都需要回答：“有限的资源如何分配给竞争的需求？”

第二，都提供隔离与安全 OS通过地址空间隔离进程，一个进程崩溃不影响其他进程[?]；Agent运行时通过沙箱隔离子Agent，一个子Agent的错误操作不影响主任务[?, ?]。两者都面临权限控制问题——“谁能做什么”。

第三，都定义标准接口 OS通过系统调用（POSIX接口）标准化应用程序与硬件的交互；Agent运行时通过MCP等协议标准化模型与外部工具的交互[?]。

5.4.4 本质差异

第一，Agent“进程”的行为不可预测 传统OS调度的进程，其行为由确定性代码定义，操作系统可以精确预测执行时间和资源需求。Agent的行为由概率式推理决定，同样的任务可能走上完全不同的执行路径——一次成功，另一次可能陷入死循环。这给调度和资源管理带来了根本性挑战。

第二，共享状态更脆弱 多进程共享内存时，一致性由硬件缓存协议（如MESI）和软件锁机制保证——精确、可验证。多Agent共享语义状态（如“对同一份文档的理解”）时，一致性只能通过自然语言沟通或外部同步机制维护——模糊、易出错。

第三，接口定义不严格 POSIX接口的每个系统调用都有精确定义的语义、参数类型、错误码和边界条件。Agent的工具调用接口虽然可以用JSON Schema描述参数，但工具的“语义行为”（什么情况下返回什么结果）远未达到系统调用级别的精确度。

这类比最有价值之处，是让许多Agent问题可以被重新表达为OS问题：

- 多Agent协作像多线程还是多进程？[?]
- 工具调用算不算系统调用？
- MCP server是否相当于设备驱动或总线协议？[?]
- 危险命令为何需要capability-based权限？[?]
- 长任务如何通过日志和中断恢复？[?]

安全层面的深刻类比 在安全层面，这一类比尤为深刻。CaMeL将操作系统能力安全原则应用于LLM Agent，创建了独立于LLM的保护层[?]。IronClaw明确提出要像操作系统一样保护AI Agent[?]。系统安全研究者开始用“agentic computing”这一新术语来定义Agent安全的基础问题[?]。这些工作说明，安全边界首先是“运行时结构问题”，其次才是提示词问题——正如OS安全首先取决于权限模型和隔离机制，其次才取决于应用程序的编码质量。

5.5 工具总线与Agent互联

5.5.1 经典计算机中的I/O系统

在经典计算机中，I/O系统负责连接CPU/内存与外部世界。它的演化历史极具启发性：**早期阶段**（1960s-1970s），每个外设需要专用接口和驱动——显卡有自己的总线，硬盘有自己的控制器，打印机有自己的并行口。**标准化阶段**（1990s-2000s），PCI/PCIe统一了主板级总线，USB统一了外设连接——硬件厂商只需遵循统一协议，设备即可即插即用。**网络化阶段**（2000s至今），TCP/IP协议栈统一了网络通信——任何设备都可以通过标准协议互联。

这一历史的核心教训是：标准化总线协议是生态系统繁荣的前提。

5.5.2 大模型系统中的对应物

在模型原生计算系统中，I/O层面正在经历类似从“点对点”到“标准化总线”的转变。

早期阶段（方法层面）：ReAct将reasoning与acting交织起来[?]; Toolformer让模型学会何时调用API[?]; HuggingGPT把LLM放到“controller”位置来调度异构模型[?]; Gorilla聚焦API调用的准确性与文档更新适应性[?]。这些工作相当于早期计算机中“直接操作外设”的阶段：每个工具都需要单独的接口和调用约定，缺乏统一的协议标准。

标准化阶段（协议层面）：MCP（Model Context Protocol）将连接AI应用与外部数据源、工具、工作流标准化[?]，扮演类似PCIe总线或USB在传统计算机中的角色——它定义了统一的发现、连接、调用和错误处理机制，使新工具可以“即插即用”。Google的A2A（Agent-to-Agent）协议则定位于agent-to-agent的互联标准[?]，支持capability discovery、task lifecycle和多模态协同，扮演类似网络协议栈中TCP/IP的角色——它使不同供应商开发的Agent可以互相发现 and 协作。

把两者合起来看，MCP是model到系统的**纵向总线**（Agent→工具），A2A是agent到agent的**横向互联**（Agent→Agent）。正如一篇综述所指出的，MCP、A2A与ACP等协议正在形成Agent互操作的基础设施层[?]

5.5.3 为什么相似

第一，都解决“接口多样性”问题 PCIe统一了主板级设备接口之前，每个设备需要专用驱动；MCP统一了工具接口之前，每个工具需要专门的API适配代码。两者都通过“定义标准协议、隐藏实现差异”来降低系统集成成本。

第二，都有分层协议栈 传统I/O有物理层（电气信号）→链路层（PCIe事务层）→协议层（NVMe/SCSI）→应用层（文件系统）。Agent I/O也有类似分层：传输层（HTTP/SSE）→协议层（MCP/A2A消息格式）→语义层（工具schema/Agent capability描述）→应用层（具体工具调用或Agent协作）。

第三，都面临发现与协商问题 USB设备插入后需要“枚举”——主机询问设备类型、能力、驱动需求；MCP服务器连接后也需要“capability discovery”——Agent查询服务器支持哪些工具、每个工具的参数schema和副作用描述。

5.5.4 本质差异

第一，调用的确定性不同 传统系统调用（如`read(fd, buffer, size)`）的行为完全可预测——参数相同、结果相同。工具调用的结果可能是非确定的（搜索引擎返回的结果随时间变化）、异步的（长时间运行的任务需要回调）且带副作用的（发送邮件、执行代码）。

第二，错误处理的复杂度不同 PCIe设备要么正常响应，要么返回明确的错误码；MCP工具可能返回语义模糊的错误（“结果不太确定”），甚至返回看似正确但实际有误的结果。这要求Agent I/O层引入**验证和冗余机制**——对关键操作进行结果校验，类似于分布式系统中的quorum读取。

5.6 多Agent协作：语义版分布式系统

5.6.1 经典计算机中的分布式系统

当单台计算机的计算能力不足以完成任务时，需要多台计算机通过网络协作，这就是分布式系统。分布式系统的核心挑战包括：**任务分解与调度**（把大任务拆成小任务，分配到不同节点）、**状态同步与一致性**（多个节点如何就共享状态达成一致）、**故障检测与恢复**（某个节点崩溃后系统如何继续运行）、**死锁避免**（多个节点互相等待如何打破僵局）、**负载均衡**（如何让所有节点都保持忙碌而非某些节点过载而其他空闲）[?]。

5.6.2 大模型系统中的对应物

当多个Agent协作完成复杂任务时，系统呈现出分布式计算的特征。

AutoGen把多agent会话作为通用基础设施[?]——多个Agent像分布式节点一样通过消息传递协作。MetaGPT用assembly line和SOP将多agent协作显式流程化[?]——类似于工业流水线式的任务分解，每个Agent负责一个环节（需求分析→设计→编码→测试）。Internet of Agents从“互联网”而非单机系统来想象agent生态[?]——强调agent integration protocol、动态组队和分布式环境。

从体系结构视角看，存在清晰的对应关系：单Agent对应单进程，多Agent协作对应多线程或多进程，跨节点的Agent集群对应分布式系统。由此引入经典的分布式系统问题：任务分解与调度、状态同步与一致性、故障检测与恢复、死锁避免、负载均衡。

5.6.3 为什么相似

第一，都面临“通信开销vs. 并行收益”的权衡 分布式系统中，Amdahl定律告诉我们并行加速比受限于串行部分；多Agent系统中，定律III（Agent加速比定律）预测类似的行为——当编排效率 E 不够高时，单纯增加Agent数量收益递减。AutoGen的多Agent实验验证了这一预测：Agent数量从2增加到8时，任务完成时间的减少幅度逐渐收窄[?]。工业案例进一步表明，协调拓扑对 E 有显著影响：Fountain采用“中心辐射”拓扑（一个中心Copilot协调筛选、文档生成、情感分析三个子Agent）实现了50%更快的候选人筛选速度和72小时内的全面配置[?]；MetaGPT采用流水线拓扑（SOP驱动的顺序协作）；AutoGen采用对等拓扑（对话式协调）。不同的拓扑在不同的任务场景下产生不同的 E 值，这暗示定律III中的 E 不是固定的标量，而是与协调结构相关的设计变量。

第二，都需要协调机制 分布式系统用共识协议（如Raft[?]）让多个节点就决策达成一致；多Agent系统需要“辩论”“投票”或“层级裁决”等机制让多个Agent就结论达成一致。

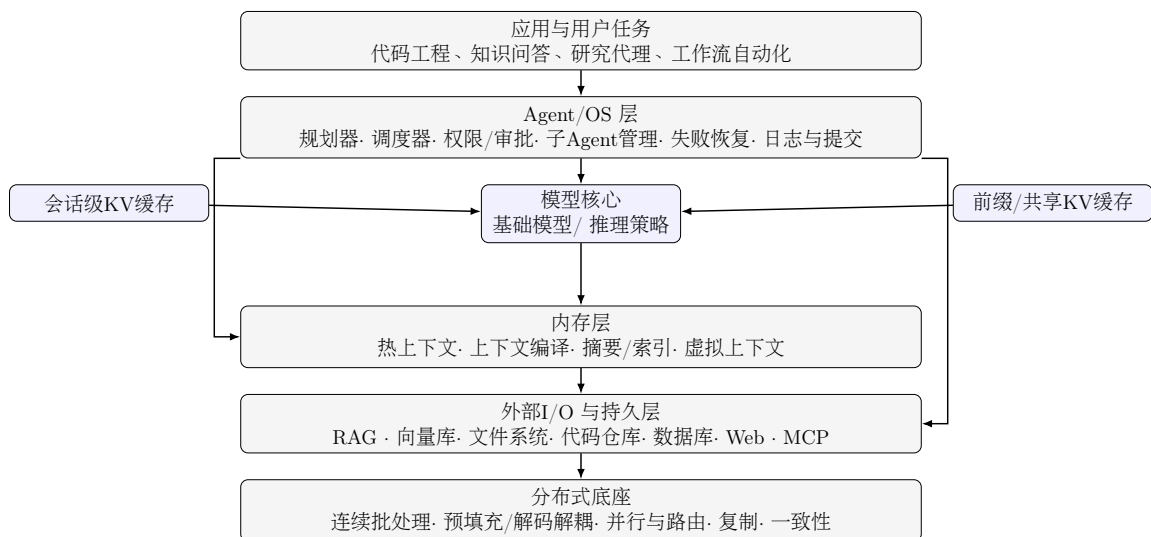


Figure 5: 面向未来模型原生计算的分层架构示意图。自上而下：应用层、Agent/OS层、模型核心与KV缓存层、内存层、I/O层、分布式底座。旁路箭头表示确定性控制平面直接管理内存层和I/O层

第三，都面临一致性与可用性的权衡 CAP定理指出分布式系统不能同时满足一致性、可用性和分区容错性[?]; 多Agent系统中，Agent之间的“语义一致性”（对同一事实的理解是否相同）与“响应速度”（是否等所有Agent都完成推理再给出结论）之间存在类似的张力。

5.6.4 本质差异

第一，通信带宽与精度的根本差异 分布式节点之间通过结构化协议通信（如gRPC的protobuf），消息语义精确无损；Agent之间通过自然语言或半结构化消息通信，信息在传递过程中可能被误解、遗漏或曲解——“语义噪声”是Agent分布式系统的独特挑战。

第二，故障模式的差异 分布式节点的故障通常是二元的（在线/离线）；Agent的“故障”可能是渐变的——它没有崩溃，但推理质量下降、偏离任务目标、产生幻觉。检测“Agent是否在正确执行任务”远比检测“节点是否存活”困难。

第三，组合涌现的不确定性 分布式系统的行为是各节点行为的确定组合；多Agent系统的行为可能产生**涌现效应**——多个Agent各自正确执行，但整体产出不可预测的结果。这对系统的可调试性和可问责性提出了全新要求。

图 ??给出本文建议的模型原生计算分层架构示意。图的每一层对应本文分析的一个组件层次：模型核心（CPU）→ KV缓存（缓存）→ 内存层（虚拟内存）→ Agent/OS层（操作系统）→ I/O与持久层（外部设备）→ 分布式底座（集群）。注意Agent/OS层通过旁路直接访问内存层和I/O层，体现了双平面架构中确定性控制平面管理资源的职责。

图 ??给出一个典型请求的控制流。该流程体现了双平面架构的交互方式：概率平面负责生成和理解，确定性平面负责调度、审批和状态管理。

6 智能计算体系结构模型：ICAM

前四节的类比框架提供了直觉，但直觉本身不足以指导工程设计。经典计算机体系结构之所以能持续演进八十余年，关键在于它拥有**形式化的模型**：从指令集架构（ISA）的软硬件契约，到存储层次的局部性理论，再到Amdahl定律的量化分析框架。本节将前文的类比**形式化**为智能计算体系结构模型（Intelligent Computing Architecture Model, ICAM），包含六个功能层的分层定义、层间接口契约和六条设计公理，为后续的量化分析奠定基础。

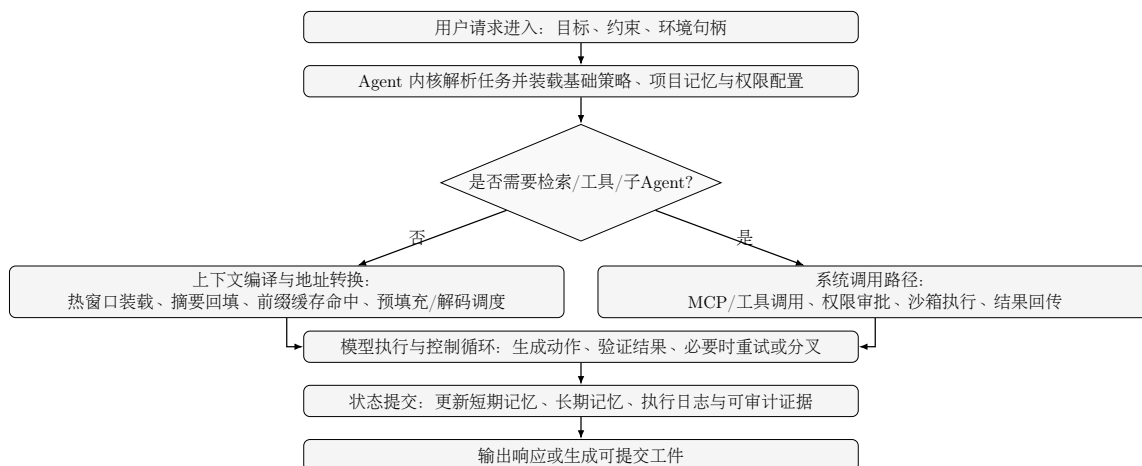


Figure 6: 模型原生计算请求的交互流程图。菱形为确定性控制平面决策，矩形交替包含概率执行与确定性控制

6.1 ICAM六层定义

经典计算机体系结构的核心思想是**分层抽象**：硬件隐藏在ISA之后，操作系统通过系统调用提供服务，应用程序只需调用库函数。每一层只关心相邻层的接口，无需了解远层的实现细节——这使得整个系统的复杂性可控、可组合。ICAM将这一思想迁移到模型原生计算领域，定义了六个功能层。

定义6.1 (智能计算体系结构模型ICAM). 智能计算体系结构模型 (*Intelligent Computing Architecture Model, ICAM*) 将模型原生计算系统定义为六个功能层，每层通过明确的接口契约与相邻层交互：

1. **L1 物理执行层**：GPU/TPU/ASIC/存算一体硬件，执行矩阵运算和注意力计算。这一层相当于经典体系结构中的ALU和浮点单元——它只负责执行计算指令，不关心计算的语义含义。例如，NVIDIA H100的Transformer引擎在硬件层面加速FP8矩阵乘法，即属于L1的范畴。
2. **L2 推理服务层**：模型权重加载、KV缓存管理、连续批处理 (*continuous batching*)、预填充/解码调度。这一层相当于经典体系结构中的微架构和缓存层次——它管理计算资源与热点数据，使得上层的“调用”尽可能高效。*vLLM*通过PagedAttention实现KV缓存的按需分配与共享[?], *SGLang*通过RadixAttention实现前缀缓存复用[?], 它们都是L2的代表实现。
3. **L3 上下文管理层**：热/温/冷记忆分层、上下文编译、摘要与检索、语义虚拟内存。这一层相当于经典体系结构中的虚拟内存子系统——它将有限的物理上下文窗口虚拟化为更大的逻辑工作空间。*MemGPT*提出“虚拟上下文管理”，将LLM的上下文窗口类比为内存、外部存储类比为磁盘，通过LLM自主管理记忆的换入换出[?]; *MemoryOS*借鉴操作系统的内存管理机制设计了分层记忆架构[?]——它们都是L3的代表实现。
4. **L4 语义接口层**：工具schema (Tool ABI)、记忆API (Memory API)、追踪API (Trace API)。这一层相当于经典体系结构中的系统调用接口和ABI——它定义了智能系统访问外部能力的统一协议。*MCP* (Model Context Protocol) 定义了模型与工具之间的标准化通信协议[?], *A2A* (Agent-to-Agent) 定义了Agent之间的互操作协议[?]/——它们是L4的代表实现。
5. **L5 编排层**：任务规划、Agent调度、权限审批、失败恢复、状态提交。这一层相当于经典体系结构中的操作系统内核——它负责任务的调度、资源分配、安全隔离和容错。*OpenAI Codex*将Agent任务提交到沙箱中执行，支持子Agent的创建与协调[?, ?]; *Claude Code*通过skill机制扩展Agent能力，通过MCP连接外部工具[?, ?]——它们是L5的代表实现。
6. **L6 应用层**：用户任务、Agent应用、工作流自动化。这一层相当于经典体系结

构中的用户应用——它承载最终用户的需求。*SWE-agent*自动化地解决*GitHub issue*，*OpenHands*作为通用软件开发Agent——它们是L6的代表实现。值得注意的是，L6的用户群体正在从专业工程师扩展到领域专家：*Anthropic*法律团队使用*Claude*将营销审核周转从2-3天缩短到24小时；*Zapier*在89%的组织范围内实现了AI采用，部署了800多个内部Agent。这一趋势表明L6需要支持更丰富的交互模式和更强的内置护栏。

ICAM的分层并非任意的拆分，而是遵循一个基本原则：每一层为上层提供服务，同时依赖下层的接口——恰如网络协议栈或操作系统层次结构的设计哲学。这种分层的好处是**关注点分离**：L2的工程师优化KV缓存策略时无需关心Agent的编排逻辑，L5的开发者设计任务调度时无需理解注意力计算的硬件实现。

表 ?? 将ICAM各层与传统计算机体系结构做系统对比，并给出当前的代表实现。

6.2 层间接口契约

经典计算机体系结构之所以能够持续演进，关键在于**层间接口的稳定性**。ISA定义了软硬件之间的契约——Intel的CPU从8086到今天的Core Ultra，微架构发生了翻天覆地的变化，但x86 ISA保持了向后兼容；POSIX定义了应用与操作系统之间的契约——Linux内核从2.0演进到6.x，应用程序几乎无需修改。这种“接口稳定、实现可变”的原则使得每一层可以独立优化。ICAM同样为每对相邻层定义接口契约。下文逐一说明每个接口的操作、不变量和度量指标。

6.2.1 L1-L2接口：张量运算契约

L2向L1请求张量运算。用通俗的话说，这个接口就像“应用程序调用GPU驱动程序”——上层说“请帮我计算这个矩阵乘法”，下层负责高效执行并返回结果。

- **操作**：`forward(batch_tokens) → logits`。输入一批token的嵌入向量，输出对应的logits概率分布。
- **不变量**：运算精度不低于配置的最低精度（如FP16/BF16）。
- **度量**：FLOPS利用率（衡量硬件算力被利用了多少）、内存带宽利用率、TTFT（Time To First Token，首token延迟）、TPOT（Time Per Output Token，每token延迟）。

6.2.2 L2-L3接口：KV块管理契约

L3向L2请求KV块的分配、加载和释放。这个接口就像“应用程序调用`malloc/free`”——上层说“我需要一段内存来存储KV缓存”，下层负责分配物理空间、处理碎片、在容量不足时执行回收。

- **操作**：`alloc / load(prefix_hash) / evict(policy)`。分配新的KV块、按前缀哈希加载已有KV块、按策略回收不活跃的KV块。
- **不变量**：KV块不丢失当前推理步所需的状态——类似于操作系统保证不会换出正在被进程使用的内存页。
- **度量**：KV命中率（ H ，即定律I的核心参数）、显存峰值、页置换次数。

6.2.3 L3-L4接口：上下文装载契约

L4向L3请求上下文装载。这个接口就像“应用程序通过内存映射（`mmap`）加载文件”——上层说“我需要关于X的信息”，下层负责从外部存储中检索、压缩、装载到上下文窗口中。

- **操作**：`read(query) / prefetch(hint) / compact / evict`。按查询检索上下文、按提示预取、压缩上下文、回收不活跃上下文。

- **不变量**：返回内容附带来源追踪和时效标注——类似于文件系统保证返回的数据是最新版本。
- **度量**：检索召回率、上下文压缩比、加载延迟。

6.2.4 L4-L5接口：能力调用契约

L5向L4请求工具调用和记忆读写。这个接口就像"应用程序通过系统调用请求操作系统服务"——上层说"我需要读写文件"（对应记忆读写）或"我需要访问网络"（对应工具调用），下层负责权限校验和审计。

- **操作**：`call(tool, args, capability_token) / query(memory)`。带权限令牌调用工具、查询记忆。
- **不变量**：无capability token的工具调用被拒绝（类比：无权限的进程不能打开文件）；副作用操作可审计[?]
- **度量**：工具调用成功率、权限拒绝率。

6.2.5 L5-L6接口：任务提交契约

L6向L5提交任务。这个接口就像"用户通过命令行启动程序"——上层说"请帮我完成这个任务"，下层负责任务的排队、执行、监控和结果返回。

- **操作**：`submit(task_spec) / poll / cancel`。提交任务规格、轮询状态、取消任务。
- **不变量**：任务执行产生可追踪trace；失败时回滚副作用——类似于数据库的事务原子性保证。
- **度量**：任务完成率、人工接管率。

值得注意的经验约束 Anthropic的内部研究显示，工程师在工作中约60%使用AI，但只能"完全委托"0-20%的任务[?]。这一"协作悖论"意味着L5-L6接口的任务提交契约不能仅建模为二元操作（提交/不提交），而需要反映一个委托置信度谱系：任务的可验证性越高、组织上下文依赖越低，委托置信度越高。这与双平面架构直接相关——确定性控制平面提供的可审计轨迹正是提高可验证性、从而提高委托置信度的关键机制。

6.3 双平面与ICAM的交叉映射

第 ??节提出了双平面架构：概率执行平面（LLM的不确定性推理）和确定性控制平面（可审计的逻辑控制）。一个自然的问题是：双平面与ICAM六层模型是什么关系？答案是：它们是**正交的两个维度**。ICAM是纵向分层（从硬件到应用），双平面是横向切分（从概率到确定性）。它们的交叉形成了一个 6×2 的矩阵。

表 ??展示了这一交叉映射。

从表中可以看出两个关键特征：第一，**概率平面的重心偏下**，主要覆盖L1（物理执行）和L2（推理服务），并渗透到L3（语义检索与上下文编译）。这是因为LLM的核心能力——语言理解和生成——集中在这些层。第二，**确定性平面的重心偏上**，主要覆盖L5（编排层）和L4的接口契约部分（权限审批、审计、capability标注），并延伸到L3的状态管理部分（记忆保留策略、版本戳）。这是因为系统级的安全、可靠、可审计需求需要确定性保证。

L4是一个特殊的"过渡层"：工具schema和协议规范属于确定性平面（它们是形式化的接口定义），而工具结果的语义理解属于概率平面（LLM需要理解自然语言描述的返回值）。L6（应用层）是两个平面的消费者：用户同时获得概率平面的智能输出和确定性平面的可审计轨迹。

这一交叉映射解释了为什么不同文献对"LLM在哪一层"的判断不同——它们看到的是同一系统中不同平面的不同层。例如，聚焦推理优化的研究者认为LLM是L1-L2的计算引擎[?, ?]；聚焦Agent系统的研究者认为LLM是L5-L6的智能内核[?, ?]；聚焦上下文管理的研究

者认为LLM是L3的记忆处理器[?, ?]——这些视角并不矛盾，而是观察到了ICAM模型的不同层面。

6.4 设计公理体系

经典计算机体系结构八十年的设计经验可以提炼为一组**设计原则**：局部性原理指导缓存设计，抽象层次原则指导接口定义，最小权限原则指导安全设计。本文将这些原则迁移到模型原生计算领域，结合模型原生计算的特殊性，提出六条设计公理。每条公理指明其经典来源、在ICAM中的体现以及具体的设计含义。

公理6.1 局部性公理

模型原生计算呈现**时间局部性**（近期访问的上下文片段将再次被访问）和**语义局部性**（语义相近的查询访问相似的上下文）。

直观解释

正如程序在运行时倾向于反复访问同一组内存页（这催生了LRU缓存），LLM在多轮对话中也倾向于反复引用同一批上下文片段。用户先问"Python的装饰器怎么用"，后续的问题往往围绕同一主题展开——这就是语义局部性。

经典来源

缓存设计的理论基础[?]

ICAM体现

L2的KV缓存复用（相同前缀共享KV缓存）、L2的前缀缓存共享（多个请求共享公共system prompt的KV缓存）、L3的语义缓存（语义相似的查询复用之前的生成结果）。

设计含义

ICAM的每一层都应基于局部性做缓存/预取。定律I（第 ?? 节）将这一公理量化为可计算的公式。

公理6.2 分层抽象公理

复杂性应通过分层来管理：每层只依赖下层的接口契约，而不依赖其实现细节。

直观解释

正如应用程序不需要知道CPU是RISC还是CISC、缓存是4-way还是8-way associative，Agent逻辑也不需要知道底层模型是通过什么推理引擎服务的。

经典来源

ISA/微架构分离、OS内核/用户态分离[?, ?]。

ICAM体现

模型升级（L2的实现变化）不应破坏Agent逻辑（L5的功能）；工具schema变更（L4的接口调整）不应导致记忆失效（L3的数据损坏）。

公理6.3 概率执行公理

模型原生计算的核心执行（模型前向推理）是概率式的：相同输入不保证相同输出。

直观解释

这是模型原生计算与经典计算**最根本的差异**。经典计算中， $2+2$ 永远等于4；但在LLM中，同一个问题问两次可能得到不同的回答——这不是bug，而是特性。这种概率性使得传统的确定性测试方法失效：我们不能通过"回放相同的输入序列"来验证系统正确性，而需要定义"语义等价性"作为验证标准。

经典来源

无经典对应——这是模型原生计算独有的公理。

ICAM体现

整个ICAM模型的分层设计需要为概率性"留出空间"。L4-L5的接口契约不能要求工具调用的结果完全确定，而必须容忍语义层面的变化。L5的失败恢复机制不能依赖精确的输入回放，而需要语义级别的检查点。

公理6.4 虚拟化公理

有限物理资源应通过虚拟化呈现为更大的逻辑资源，同时保持隔离与保护。

直观解释

正如32位操作系统通过虚拟内存让程序以为它有4GB内存（尽管物理内存可能只有1GB），模型原生计算系统需要让Agent以为它有无限长的上下文窗口（尽管物理上下文窗口只有128K token）。

经典来源

虚拟内存、进程隔离[?]

ICAM体现

MemGPT的虚拟上下文管理[?]; PagedAttention的页式KV管理[?]——将KV缓存按"页"组织，按需分配，回收不活跃页，与操作系统的虚拟内存管理如出一辙；Agent沙箱[?]——将Agent的执行环境与宿主系统隔离，正如进程的地址空间隔离。

公理6.5 最小权限公理

每个执行单元应只被授予完成任务所需的最小权限集合。

直观解释

正如文本编辑器不需要管理员权限就能编辑文件，一个负责代码生成的Agent也不应该拥有删除整个代码仓库的权限。

经典来源

操作系统安全原则[?]

ICAM体现

CaMeL的能力安全机制[?]——为每个Agent操作分配细粒度的capability token；Codex的OS级沙箱[?]——限制Agent的文件系统和网络访问权限。

公理6.6 可观测性公理

每个有副作用的操作都必须产生可追踪、可审计的事件记录。

直观解释

正如数据库记录每一次写操作的WAL（Write-Ahead Log），Agent的每一次工具调用、文件修改都应该被记录下来，使得事后可以追溯"为什么做了这个操作"。

经典来源

系统日志、性能监控[?]

ICAM体现

OpenAI Agents SDK的tracing机制[?]——记录Agent的每一步决策和工具调用；Agent安全研究中的trace grading[?]——评估Agent操作轨迹的安全性。

7 量化定律

经典计算机体系结构的力量在于它有**可计算的原则**。Amdahl定律告诉我们：如果程序中串行部分占比例为 $(1 - f)$ ，即使把并行部分加速 p 倍，总体加速比也不会超过 $1/(1 - f)$ ——这为并行计算设定了理论上界[?]。Roofline模型告诉我们：每个计算核心的性能上限由其计算强度（Arithmetic Intensity）和内存带宽共同决定——这帮助工程师快速判断瓶颈在计算还是在访存[?]。这些定律的价值不在于它们的数学复杂度，而在于它们提供了**统一的量化语言**，使得工程师可以做出基于数据的决策。

本节为模型原生计算提出三条量化定律。在形式上，它们与Amdahl定律同构；但在语义上，它们刻画了模型原生计算特有的性能约束。本文明确承认：定律I和定律III在数学形式上并非全新的发现，而是将Amdahl定律的分析框架**有意识地迁移**到模型原生计算场景，并用场景特有参数重新解释。这种迁移的价值不在于数学创新，而在于：为原本缺乏量化语言的设计空间提供可计算的决策依据，使得"该优先优化什么"从直觉判断变为定量分析。

7.1 定律I：语义局部性定律

定律7.1 (语义局部性定律). 在自回归推理中，推理加速比 S 由KV缓存命中率 H 和KV复用加速比 α 决定：

$$S = \frac{1}{(1 - H) + H \cdot \alpha^{-1}} \quad (2)$$

7.1.1 参数的直观含义

在展开推导之前，先理解两个核心参数的物理含义：

- H (KV缓存命中率)：有多少比例的KV可以直接复用而不需要重算。想象一个客服系统，如果用户连续5轮对话都围绕同一主题，前4轮生成的KV缓存可以被第5轮直接复用，那么 H 就接近1。反之，如果每次对话都是全新话题，之前的KV缓存毫无用处， H 就接近0。 H 的值域为 $[0, 1]$ 。
- α (KV复用加速比)：命中比未命中快多少倍。当KV缓存命中时，系统只需从内存中读取已有的KV向量（时间复杂度 $O(1)$ ）；当未命中时，系统必须从头计算全部注意力（时间复杂度 $O(n^2)$ ）。 α 量化了这一差距。在典型场景下， α 的值在10到100之间。

7.1.2 完整推导

推导从基本的时间分析开始。

第一步：定义时间分解 设一次推理请求的总时间为 T ，它由两部分组成：

$$T = T_{\text{miss}} + T_{\text{hit}} \quad (3)$$

其中 T_{miss} 是KV缓存未命中部分的时间（需要从头计算）， T_{hit} 是KV缓存命中部分的时间（直接复用已有KV）。

第二步：用命中率表达各部分时间 设无缓存时完成同样推理的总时间为 T_{total} （即所有部分都需要重算）。当有缓存时，未命中比例为 $(1 - H)$ ，命中比例为 H 。由于命中部分可以直接复用KV，其时间为原来的 $1/\alpha$ （快了 α 倍），因此：

$$T = T_{\text{total}} \cdot (1 - H) + T_{\text{total}} \cdot H \cdot \frac{1}{\alpha} \quad (4)$$

整理得：

$$T = T_{\text{total}} \cdot \left[(1 - H) + \frac{H}{\alpha} \right] \quad (5)$$

第三步：定义加速比 加速比 S 是有缓存相比无缓存的性能提升倍数：

$$S = \frac{T_{\text{total}}}{T} = \frac{T_{\text{total}}}{T_{\text{total}} \cdot [(1-H) + H/\alpha]} = \frac{1}{(1-H) + H/\alpha} \quad (6)$$

这正是公式 (??)。

7.1.3 极限行为与直觉

- 当 $H \rightarrow 1$ （几乎全部命中）时： $S \rightarrow 1/(0 + 1/\alpha) = \alpha$ 。加速比上界由复用效率决定——即使100%命中，最快也只能快 α 倍。
- 当 $H \rightarrow 0$ （几乎全部未命中）时： $S \rightarrow 1/(1 + 0) = 1$ 。没有任何加速，退化为无缓存场景。
- 当 $\alpha \rightarrow \infty$ （命中瞬间完成）时： $S \rightarrow 1/(1 - H)$ 。加速比上界由命中率决定——即使命中是"免费的"，未命中的部分仍然消耗时间。

7.1.4 数值示例

假设一个LLM推理服务， $\alpha = 10$ （命中比未命中快10倍），我们来观察 H 对 S 的影响：

- $H = 0.5$ （一半命中）： $S = 1/(0.5 + 0.05) = 1.82$ 倍加速
- $H = 0.8$ （80%命中）： $S = 1/(0.2 + 0.08) = 3.57$ 倍加速
- $H = 0.95$ （95%命中）： $S = 1/(0.05 + 0.0095) = 16.8$ 倍加速

可见， H 从0.5提升到0.8带来约2倍加速，而从0.8提升到0.95带来约4.7倍加速——**边际收益递增**，这意味着在已有一定缓存基础时进一步提升命中率的回报特别大。

7.1.5 与Amdahl定律的对应

公式 (??)与Amdahl定律在数学形式上完全同构。Amdahl定律的原始形式为：

$$S_{\text{Amdahl}} = \frac{1}{(1-f) + f/p} \quad (7)$$

其中 f 是可加速部分的比例， p 是加速倍数。对应关系为： $f \leftrightarrow H$ （可加速比例即KV缓存命中比例）， $p \leftrightarrow \alpha$ （加速倍数即KV复用加速比）。

设计含义：提升推理性能有两条路径——提高 H （通过更好的缓存策略、前缀共享、语义缓存[?])和提高 α （通过KV加载优化、PagedAttention[?]、KV量化[?]）。根据公式，这两条路径具有**互补性**：当 H 已经很高时，进一步提升 H 的边际收益大；当 H 较低时，提高 α 的效果有限，应先解决缓存策略问题。

7.1.6 实证验证

数据来源：定律I的验证需要两组数据——系统报告的加速比和可以反推出的 H 值。

1. **vLLM/PagedAttention:** Kwon等人报告vLLM相比基线推理系统实现2–4×的吞吐提升[?]。取 $\alpha \approx 10$ （KV加载比重新计算快约10倍，这是一个保守估计），代入公式可得 $H \approx 0.56$ – 0.83 。这意味着vLLM的PagedAttention机制使得56%–83%的KV缓存可以被复用，而非每次从头计算。
2. **Prompt Cache:** Gim等人报告TTFT（首token延迟）降低至原来的1/8至1/60[?]，即 S 从8到60。取 $\alpha \approx 60$ （Prompt Cache的语义缓存允许直接跳过大量计算），对应 H 从0.75到接近0.98。这说明对于高度结构化的prompt（如system prompt + 用户模板），缓存命中率可以非常高。

7.2 定律II：上下文预算定律

定律7.2 (上下文预算定律). 模型的有效工作集 W_{eff} 受上下文窗口大小 C 和注意力保持率 $\beta(L)$ 的联合约束:

$$W_{\text{eff}} \leq C \cdot \beta(L) \quad (8)$$

其中 C 是模型声称的上下文窗口大小 (如 $128K$ 、 $1M$ token), L 为信息在窗口中的位置距离, $\beta(L) \in [0, 1]$ 衡量位置 L 处信息被有效利用的概率。

7.2.1 参数的直观含义

- C (上下文窗口大小): 这是模型的"物理内存容量"。例如GPT-4-Turbo的 $C = 128K$, Gemini 1.5 Pro的 $C = 1M$ 。 C 决定了模型一次性能"看到"多少token。
- $\beta(L)$ (注意力保持率): 这是定律II的核心创新。它衡量的是: **放在上下文窗口中位置 L 处的信息, 模型真正能用到多少**。 $\beta(L) = 1$ 表示该位置的信息被完美利用, $\beta(L) = 0$ 表示该位置的信息被完全忽略。
- W_{eff} (有效工作集): 模型的"真正可用记忆容量"。即使 $C = 1M$, 如果 $\beta(L)$ 在某些位置很低, 那么 W_{eff} 可能远小于 C ——就像买了一台1TB硬盘但只能稳定使用100GB。

7.2.2 为什么 $\beta(L)$ 会衰减——"中间丢失"现象

$\beta(L)$ 不是常量, 而是随位置 L 衰减的函数。这一事实来自一个被广泛观察到的现象: "中间丢失" (Lost in the Middle)。

Liu等人的开创性工作系统地研究了这一问题[?]: 他们在长上下文中不同位置放置关键信息, 然后测试模型能否检索到该信息。结果发现模型呈现出**U形曲线**——位于上下文**开头** (类似primacy effect) 和**末尾** (类似recency effect) 的信息被很好地检索到, 而位于**中间**的信息检索准确率显著下降。

后续研究进一步量化了这一衰减: RULER基准表明, 上下文从4K增长到128K时, 检索准确率从 $>95\%$ 降至 $<70\%$ [?]; LongBench v2确认深度推理任务的衰减更为严重[?]; LOFT的研究表明即使模型支持超长上下文, 在实际利用效率上也存在显著不足[?]。近期的研究指出LLM实际有效利用的上下文长度可能仅为声称窗口的10%–20%。

7.2.3 为什么使用指数衰减模型

本文采用指数衰减形式来建模 $\beta(L)$:

$$\beta(L) \approx \beta_0 \cdot e^{-\lambda L/C} \quad (9)$$

选择指数衰减有三个理由: **第一**, 它是最简单的单调衰减函数, 参数 λ 有直观的物理含义—— λ 越大, 衰减越快, 有效上下文越短。**第二**, 注意力权重本身是softmax的输出, 天然具有指数形式, 因此用指数函数建模其衰减在结构上是合理的。**第三**, RULER等基准的实验数据在log尺度上近似线性[?], 与指数衰减模型一致。

需要强调的是: λ 的精确值取决于具体模型和任务类型, 本文暂不对其做参数拟合。定律II的价值在于提供了一个**思考框架**——它告诉我们 $W_{\text{eff}} < C$, 而这个差距有多大、如何缩小, 是后续研究的问题。

7.2.4 与经典"工作集理论"的对应

定律II与Denning的工作集模型有深刻的对应关系[?]

1968年, Peter Denning提出了工作集模型: 进程在时间窗口 τ 内访问的内存页面的集合构成工作集 $W(\tau)$ 。Denning的核心洞察是: 如果物理内存小于 $W(\tau)$, 系统就会发

生thrashing（颠簸）——CPU大部分时间在等待页面换入换出，而不是在做有用的工作[?, ?]。

公式 (??) 是工作集模型的语义版本： C 对应物理内存大小， W_{eff} 对应进程的真正工作集。关键差异在于：经典工作集模型中，页面的"有用性"是二值的（进程要么访问该页面，要么不访问）；而在定律II中， $\beta(L)$ 是连续的——模型对上下文中不同位置的信息利用程度是一个渐变的谱。

7.2.5 设计含义

定律II有三条直接的设计含义：

(1) **单纯增大 C 的边际收益递减**增大上下文窗口（如从128K扩展到1M）确实增大了 C ，但如果 $\beta(L)$ 随 L 快速衰减，那么 W_{eff} 的增长远小于 C 的增长。这意味着单纯追求更大的上下文窗口不是解决"记忆不够用"的根本方案。

(2) **关键信息应放在 β 高的位置**由于 $\beta(L)$ 在上下文开头和末尾较高，在设计提示词(prompt)时应将关键信息放在这些位置，而将辅助信息压缩或外移到RAG系统中。

(3) **这为分层记忆(L3)提供了定量依据**当 C 固定时，只能通过语义虚拟内存(L3的职责)来突破 W_{eff} 的限制——将不活跃的上下文"换出"到外部存储，将活跃的上下文"换入"到 β 值高的位置。MemGPT的虚拟上下文管理[?]和MemoryOS的分层记忆[?]正是这一思路的体现。

7.2.6 实证验证

数据来源：定律II的验证依赖于"模型声称的上下文窗口 C "与"实际可用的有效上下文 W_{eff} "之间的差距。

1. **RULER基准：**Hsieh等人设计了RULER基准，通过在不同长度的上下文中放置needle-in-a-haystack式检索任务来测试模型的有效上下文[?]。结果表明，即使模型声称支持128K上下文，在复杂检索任务上的准确率从4K时的>95%下降到128K时的<70%。这意味着 $W_{\text{eff}} \approx 0.7 \times 128K \approx 90K$ ，远小于声称的 C 。
2. **LongBench v2：**Bai等人发现深度推理任务（如数学证明、多步逻辑）的衰减速度远快于简单检索任务[?]。这说明 λ （衰减系数）不是模型的固定属性，而是与任务类型相关的。

7.3 定律III：Agent加速比定律

定律7.3 (Agent加速比定律)。多Agent并行处理时，加速比 S_{agent} 受可并行化比例 F 、子Agent数量 N 和编排效率 E 约束：

$$S_{\text{agent}} = \frac{1}{(1 - F) + \frac{F}{N \cdot E}} \quad (10)$$

7.3.1 参数的直观含义

- F （可并行化比例）：任务中有多少比例可以分给多个Agent同时执行。例如，一个"对比三篇论文"的任务，三篇论文的摘要可以分别由三个Agent同时提取（ F 较高）；而"按照严格的先后顺序编译、测试、部署"的任务，大部分步骤必须串行执行（ F 较低）。 F 的值域为 $[0, 1]$ 。
- N （子Agent数量）：系统中可同时调度的Agent数量。类似于分布式系统中的处理器数量。例如，Codex支持为复杂任务创建多个子Agent[?]，Claude Code支持通过sub-agent机制并行处理子任务[?]。
- E （编排效率）：协调多个Agent的开销因子， $E \in (0, 1]$ 。 E 反映了Agent编排的额外成本：上下文同步（Agent之间需要共享中间结果）、结果合并（多个Agent的输出需要被

整合)、冲突检测(多个Agent可能修改同一资源)和权限协调(每个Agent的权限需要独立审批)。 $E = 1$ 表示无编排开销, $E < 1$ 表示存在开销。

7.3.2 完整推导

推导过程与定律I类似, 但引入了编排效率的概念。

第一步: 定义时间分解 设任务在单个Agent上的执行时间为 T_{total} 。任务分为可并行部分(比例 F)和串行部分(比例 $1 - F$)。

第二步: 计算各部分时间

- 串行部分的时间: $(1 - F) \cdot T_{\text{total}}$ (无法加速)。
- 并行部分在 N 个Agent上执行, 但存在编排开销 E , 因此实际加速倍数为 $N \cdot E$ 而非 N 。时间: $F \cdot T_{\text{total}} / (N \cdot E)$ 。

第三步: 汇总并计算加速比

$$T = (1 - F) \cdot T_{\text{total}} + \frac{F \cdot T_{\text{total}}}{N \cdot E} \quad (11)$$

$$S_{\text{agent}} = \frac{T_{\text{total}}}{T} = \frac{1}{(1 - F) + \frac{F}{N \cdot E}} \quad (12)$$

第四步: 验证退化情况当 $E = 1$ (无编排开销) 时, 公式退化为标准Amdahl定律, 验证了推导的一致性。

7.3.3 极限行为与直觉

- 当 $N \rightarrow \infty$ 时: $S_{\text{agent}} \rightarrow 1/(1 - F)$ 。加速比上界由任务的串行比例决定——无论增加多少Agent, 串行部分都无法被并行化。
- 当 $F \rightarrow 1$ (几乎全部可并行) 时: $S_{\text{agent}} \rightarrow N \cdot E$ 。加速比由Agent数量和编排效率决定。
- 当 $E \rightarrow 0$ (编排开销极大) 时: $S_{\text{agent}} \rightarrow 1/(1 - F + F/\infty) \rightarrow 1/(1 - F)$ 。即使有很多Agent, 如果编排效率极低 (Agent之间大部分时间花在协调上), 最终效果接近于"每个Agent独立工作然后等串行部分完成"。

7.3.4 数值示例

假设一个代码审查任务, $F = 0.6$ (60%的审查工作可以分配给多个Agent), $E = 0.7$ (编排效率70%, 有30%的开销用于协调):

- $N = 2$: $S = 1/(0.4 + 0.6/(2 \times 0.7)) = 1/(0.4 + 0.43) = 1.20$ 倍
- $N = 4$: $S = 1/(0.4 + 0.6/(4 \times 0.7)) = 1/(0.4 + 0.21) = 1.64$ 倍
- $N = 8$: $S = 1/(0.4 + 0.6/(8 \times 0.7)) = 1/(0.4 + 0.11) = 1.96$ 倍
- $N = 16$: $S = 1/(0.4 + 0.6/(16 \times 0.7)) = 1/(0.4 + 0.054) = 2.20$ 倍

可见, Agent数量从2增加到4带来约37%的额外加速, 而从8增加到16仅带来约12%的额外加速——**收益逐渐收窄**, 这正是Amdahl定律的核心特征。

工业实证校验 以Fountain的Copilot系统为例[?]: 其中心编排Agent协调候选人筛选、文档生成和情感分析三个专业子Agent ($N = 3$)。如果基线流程需要168小时 (一周) 完成招聘中心的全面配置, 而实际在72小时内完成, 则 $S_{\text{agent}} = 168/72 = 2.33$ 。代入公式反推, 当 $N = 3$ 时, 合理的参数估计为 F 约0.7, E 约0.65。这一实证校验与上文 $F = 0.6, E = 0.7$ 的假设在数量级上一致, 表明定律III的参数空间可以由工业数据进行标定。

7.3.5 与经典Amdahl定律的对比

定律III与Amdahl定律的区别在于引入了编排效率 E :

$$S_{\text{Amdahl}} = \frac{1}{(1-F) + F/N} \quad \text{vs.} \quad S_{\text{agent}} = \frac{1}{(1-F) + F/(NE)} \quad (13)$$

E 的存在使得Agent系统的加速比总是低于经典Amdahl定律的预测。这是因为: 在经典并行计算中, 处理器间的通信开销通常可以通过硬件(如高速互联网络)和算法(如减少通信的算法设计)来控制; 但在Agent系统中, 编排开销不仅包含通信成本, 还包含**LLM推理本身的概率性**——Agent之间需要用自然语言协调, 而自然语言通信天生比二进制协议低效且容易产生误解。

7.3.6 设计含义

定律III有三条直接的设计含义:

(1) **盲目增加Agent数量不一定提升效率**根据公式, 当 E 较低时(编排开销大), 增加 N 的收益很小。这解释了为什么简单的"堆Agent"策略往往不如预期有效。

(2) **应优先提高 F 和 E** 提高 F (更好的任务分解, 将更多串行步骤转化为可并行的子任务)和提高 E (更低的编排开销, 通过标准化的Agent间通信协议如A2A[?]来减少协调成本)的投入回报率远高于单纯增加 N 。

(3) **存在最优的Agent数量**对于给定的 F 和 E , 存在一个 N^* 使得增加更多Agent的边际收益低于其边际成本(每个Agent都需要消耗LLM推理资源)。 N^* 的估计可以从公式导出: 当 $\partial S/\partial N$ 低于某个阈值时, 不应再增加Agent。

7.3.7 实证验证

数据来源: 定律III的验证需要多Agent系统的实际性能数据。

1. **AutoGen:** Wu等人报告了多Agent系统的实验数据, Agent数量从2增加到8时, 任务完成时间的减少幅度逐渐收窄[?]
——这与定律III的预测一致: 随着 N 增加, S_{agent} 的增长率递减。
2. **GTA-2基准:** Wang等人报告开放工作流基准GTA-2的测试结果, 顶尖模型的开放工作流成功率仅14.39%[?]。这暗示当前系统的 F 和 E 都较低: 任务分解能力不足导致 F 小(大部分工作仍是串行的), Agent间协调效率低导致 E 小。
3. **Language Model Teams:** 近期的arXiv论文将多Agent LLM团队类比为分布式系统, 并直接用Amdahl定律框架分析其加速比[?], 发现高度可并行化的任务(如独立文档摘要)能获得接近线性的加速, 而强依赖型任务(如多步推理)的加速比接近1——这与定律III中 F 的作用完全一致。

7.4 三条定律的统一视角

表 ?? 汇总了三条定律的核心公式、支持数据和待验证问题。

图 ?? 绘制了三条量化定律的特征曲线。

三条定律的共同特征是: 它们都揭示了模型原生计算系统中的**性能上界**, 并指出了突破上界的方向。定律I告诉我们KV缓存策略的理论极限; 定律II告诉我们扩大上下文窗口的实际收益; 定律III告诉我们增加Agent数量的有效边界。这三条定律共同构成了ICAM模型的**量化骨架**, 将第 ?? 节的定性设计原则转化为可计算的工程指标。

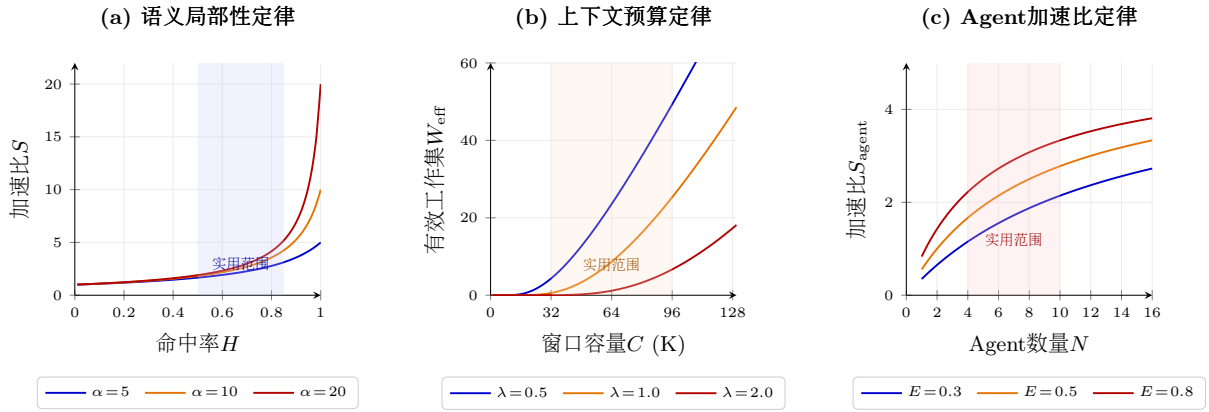


Figure 7: 三条量化定律的特征曲线。(a) 语义局部性定律: $S = 1/((1 - H) + H/\alpha)$ 。(b) 上下文预算定律: $W_{\text{eff}} = C \cdot \beta_0 e^{-\lambda C_0/C}$ 。(c) Agent加速比定律: $S = 1/((1 - F) + F/(NE))$, 取 $F = 0.8$ 。阴影区域为典型工作范围。

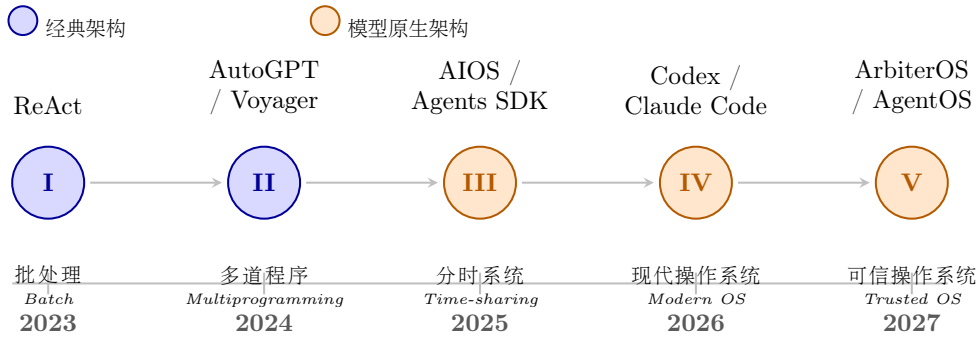


Figure 8: Agent框架的五代演进时间线（2023–2026）

8 Agent框架演进：从ReAct到智能操作系统

Agent框架（Agent Framework）是指围绕大语言模型构建的、使其能够自主规划、调用工具、管理状态并完成复杂任务的软件系统。从2022年的研究原型到2025年的产品化运行时，Agent框架正在经历一次类似操作系统从单任务批处理到多任务分时、从无保护到有保护、从无治理到可信治理的系统性演进。本节梳理这一演进的五个阶段，每个阶段从**核心架构特征**、**代表性系统及其具体做法**、以及**与经典操作系统演进的对应关系**三个维度展开分析。

图 ??展示了Agent框架五代演进的时间线。

第一代：执行范式确立（2022–2023）

核心特征 第一代Agent框架确立了“推理–行动–观察”交织循环（Thought–Action–Observation loop）的基本范式。在此之前，大语言模型的使用方式是“单轮问答”：用户给出提示，模型一次性返回回答，类似于向批处理系统提交一个作业并等待输出。第一代框架打破了这一限制：模型不再是被动回答者，而是在循环中主动**推理**（分析当前状态并决定下一步）、**行动**（调用工具或执行操作）、**观察**（读取工具返回的结果），然后进入下一轮循环。

代表工作 ReAct[?]是这一代的奠基工作。它的核心做法是在提示模板中引入“Thought”字段，要求模型在每次调用工具前先显式输出推理过程。例如，当用户问“莎士比亚最长的剧本是什么？”时，模型不是直接猜测答案，而是先输出“我需要搜索莎士比亚所有剧本的长度”，然后调用搜索工具，再根据搜索结果推理出最终答案。这种“先想再做”的模式看似简单，却解决了大模型直接生成工具调用时常见的“幻觉式工具调用”问题。Toolformer[?]则从另一个角度切入：让模型**自学**何时以及如何调用工具，通过在训练数据中注入工具调用示例来实现。

与OS演进的对应 这一代框架类似于1950年代的单任务批处理系统（如IBM 704的Fortran Monitor System）：系统一次只处理一个任务，做完再做下一个，没有并发、没有中断、没有状态保持。ReAct的循环就是最简单的“取指-译码-执行”流水线，每次循环对应一个“指令周期”。系统没有任何资源管理能力：没有任务调度（只有一个任务）、没有内存管理（上下文窗口就是全部“内存”）、没有I/O抽象（工具调用是硬编码的）。

第二代：持续多步代理（2023）

核心特征 第二代框架的核心突破是**持续性**（persistence）和**自主目标分解**（autonomous goal decomposition）。第一代框架中，每个用户提问触发一个独立的推理-行动循环；循环结束后，所有中间状态消失。第二代框架允许Agent持续运行，自主将高层目标分解为子任务序列，维护跨步骤的中间状态，并在子任务之间传递信息。这意味着Agent开始具备“工作记忆”的能力。

代表工作 AutoGPT[?]是第二代最知名的尝试。用户给定一个高层目标（如“调研量子计算领域的最新进展并撰写报告”），AutoGPT自主将其分解为搜索论文、提取关键信息、组织大纲、撰写报告等子任务，并依次执行。它引入了长期记忆（基于向量数据库）来存储跨步骤的中间结果，使Agent能“记住”之前做过什么。然而，AutoGPT也暴露了第二代的核心问题：缺乏失败恢复机制，一旦某个子任务出错，整个链路容易陷入死循环或累积错误。Voyager[?]在Minecraft环境中做出了关键改进：引入了**技能库**（skill library）——将成功完成的子任务封装为可复用的“技能”，类似函数定义；以及**自动课程**（automatic curriculum）——根据当前能力自动生成下一步应学习的任务，类似教学大纲。这使得Agent能够持续积累经验，而不是每次都从头开始。BabyAGI则实现了更简洁的任务队列管理：维护一个待办任务列表，按优先级排序执行，并根据执行结果动态生成新任务。

与OS演进的对应 这一代框架类似于1960年代的多道程序设计（multiprogramming）：系统开始管理复杂的多步状态，任务之间可以共享和传递信息。AutoGPT的任务队列就是最原始的“作业调度器”；Voyager的技能库类似于“可执行程序库”；BabyAGI的优先级排序类似于早期的“作业调度策略”。但与成熟的OS相比，第二代框架缺少三个关键机制：**隔离**（不同任务之间可能相互干扰）、**调度**（没有抢占式调度，一个任务卡住会阻塞整个系统）、和**保护**（一个任务出错可能污染其他任务的状态）。

第三代：通用运行时（2023–2024）

核心特征 第三代框架开始系统性地引入**调度**、**隔离**和**资源管理能力**，使Agent框架从“单脚本自动化工具”升级为“通用运行时平台”。关键标志是：Agent不再只是一个串行执行的循环，而是可以被调度、被暂停、被恢复的“进程”；多个Agent可以并发运行且互不干扰；系统开始显式管理计算资源（推理次数、上下文空间、工具调用频率）。

代表工作 AIOS[?]构建了完整的Agent OS内核，其架构包含六个子系统：Agent Scheduler（负责多Agent的并发调度）、Context Manager（管理不同Agent的上下文隔离与共享）、Memory Manager（分层记忆管理）、Tool Manager（工具注册与权限控制）、IO Manager（输入输出管理）和Knowledge Manager（知识库管理）。每个Agent被封装为一个“进程”，拥有独立的状态空间和资源预算。OpenAI Agents SDK[?]则采取了更工程化的路线：将orchestration（编排）、state management（状态管理）、approvals（审批流）、handoffs（Agent间任务移交）与tracing（全链路追踪）做成代码优先（code-first）的运行时。开发者可以用Python代码定义Agent的行为规范、权限边界和协作流程，而不是用提示模板来隐式地“编程”。AutoGen[?]提出了多Agent对话框架：多个Agent通过结构化的消息传递来协作完成复杂任务，每个Agent扮演不同角色（如程序员、测试员、项目经理），类似于多进程协作。

与OS演进的对应 这一代框架类似于1960–70年代的分时操作系统（Time-Sharing OS，如Multics和Unix）。分时OS的核心突破是为每个用户（进程）提供独立的地址空间和公平的CPU时间片，使得多用户可以安全地共享同一台计算机。AIOS的Agent Sched-

uler对应分时OS的进程调度器；Context Manager对应内存管理单元（MMU）的地址隔离；Tool Manager的权限控制对应Unix的文件权限系统。Agents SDK的tracing机制则对应OS的strace/auditd系统调用追踪。

第四代：工程化操作系统（2024–2025）

核心特征 第四代框架的核心标志是面向真实软件工程环境的深度整合。前三代框架主要在实验室环境或受限场景中验证概念；第四代框架直面真实的代码库、真实的终端环境、真实的文件系统和真实的外部服务。这意味着系统必须处理前几代可以回避的问题：大型代码仓库的上下文远超模型窗口、工具调用可能产生不可逆的副作用、并发编辑可能引发冲突、真实项目需要权限和审计。

代表工作 OpenAI Codex[?]是一个运行在隔离沙箱中的云端代码Agent：它在独立的容器中checkout代码仓库，执行读写和运行命令，完成后提交差异（diff）供人类审查。Codex引入了子Agent机制（subagents）[?]：主Agent可以将子任务分派给专门的子Agent执行，子Agent在独立的沙箱中运行，结果经审批后合并——类似于Unix的fork()创建子进程。Codex还支持技能包（skills）[?]和项目级指令文件（AGENTS.md）[?]，实现了跨项目的可复用配置。

Claude Code[?]采取了互补的设计哲学：它运行在开发者本地终端，通过MCP协议[?]连接外部工具，默认只读（除非用户显式授权写入），并支持自定义hooks来拦截和审计每一次工具调用[?]。Claude Code的记忆系统[?]区分了项目级记忆（CLAUDE.md，类似于系统配置文件）和用户级记忆（跨项目的个人偏好，类似于用户home目录下的点文件）。

OpenHands[?]提供了开放的Agent平台，支持多种模型后端和沙箱环境，其OpenHands Index已成为评估代码Agent能力的标准基准之一。OSWorld[?]则从评估角度推动了这个方向：它构建了真实的操作系统环境（包含文件管理器、浏览器、终端等），要求Agent在真实的GUI环境中完成开放式任务，暴露了Agent在真实世界复杂性面前的局限性。Devin[?]进一步展现了第四代的工程化特征：作为一个“AI软件工程师”，它集成了代码编辑器、浏览器和终端，能够自主规划、编码、调试和部署。

与OS演进的对应 这一代框架类似于1980–90年代的现代操作系统（如Unix System V、Linux早期版本、Windows NT）。现代OS的核心特征是处理真实世界的复杂性：多任务调度（CFS调度器[?])、虚拟内存与按需分页、网络协议栈、设备驱动框架、安全审计。Codex的沙箱对应OS的进程隔离（每个子Agent在独立容器中运行，类似chroot或容器技术）；Claude Code的hooks对应Linux的LSM（Linux Security Modules）安全钩子；Codex的审批机制对应Unix的sudo权限提升；OpenHands的多模型后端对应OS的硬件抽象层（HAL）。

第五代：治理化操作系统（2025–至今）

核心特征 第五代框架的核心突破是将安全和治理从事后补丁升级为体系结构的第一性设计原则。前四代框架的安全措施是“加法式”的：先让Agent能工作，再加沙箱、加审批、加日志。第五代框架反过来：先定义安全边界和治理规则，再在边界内赋予Agent行动能力。这种范式的转变类似于操作系统从“可用”到“可信”的转折——正如Multics在1960年代就引入了环保护（ring protection）和强制访问控制（mandatory access control），但直到SELinux和Capability-based Security普及后，安全才真正成为OS体系结构的有机组成部分[?]。

代表工作 ArbiterOS[?]明确提出了“Probabilistic CPU + Deterministic Governor/Kernel”的分层架构：底层的LLM是概率式处理器，负责“能做什么”；上层的governor是确定性控制器，负责“应该做什么”。governor通过策略文件（policy specification）定义Agent的行为边界，类似于OS内核通过系统调用表和权限位定义进程的能力范围。

AgentOS[?]引入了受结构化操作系统逻辑（structured operating system logic）约束的推

理内核（reasoning kernel），从token级别的上下文管理出发，逐步构建出系统级的智能行为。其核心思想是：Agent的推理过程不应该是不可控的“黑箱循环”，而应该像OS内核一样，通过结构化的抽象和接口来管理。

UFO²[?]面向桌面环境，将Desktop AgentOS组织为三层结构：HostAgent（顶层协调者，类似于init进程）、AppAgent（每个应用程序一个，类似于服务进程）、和GUI-API action layer（底层操作接口，类似于驱动层）。这种分层使得Agent对不同应用程序的操作可以被独立管理和审计。

Aura[?]则在移动端实现了安全优先的Agent OS架构：System Agent（系统级Agent，拥有最高权限）、Sandboxed App Agents（每个应用一个，运行在沙箱中，类似于Android的应用沙箱）、和Agent Kernel（Agent内核，负责权限仲裁和资源分配）。Aura特别强调了移动端环境的独特风险：语义输入污染（通过恶意输入劫持Agent行为）、记忆taint传播（被污染的记忆影响后续所有操作）、以及跨应用权限泄露。

CaMeL[?]从安全原则出发，提出了基于能力（capability）的安全模型来防御提示注入攻击。其核心思想借鉴自操作系统中的capability-based security：Agent不能直接执行任何有副作用的操作，而必须通过一个确定性的capability manager来获取“临时通行证”。每个通行证只授权一个特定操作（如“只读文件X”），而不是授予全局权限。这类似于Unix从“root/非root”的粗粒度权限模型，向POSIX capability和SELinux的细粒度权限模型的演进。

IronClaw[?]系统性地论证了为什么AI Agent需要像操作系统一样被保护。它指出，传统软件安全假设“代码是确定性的、行为是可预测的”，但Agent的行为由概率式模型驱动，攻击面从“代码漏洞”扩展到了“语义漏洞”——攻击者可以通过自然语言提示来劫持Agent的行为，而不需要利用任何代码缺陷。

OWASP在2025年底发布的**Agentic Applications Top 10**[?]进一步系统化了这些威胁：Agent Goal Hijack（目标劫持）、Tool Misuse & Exploitation（工具滥用）、Identity & Privilege Abuse（身份与权限滥用）等十大风险类别，为第五代框架的安全设计提供了分类学基础。

与OS演进的对应 这一代框架对应操作系统从“可用”到“可信”的转折期。在经典OS历史上，Unix V6（1975年）是一个“可用但不安全”的系统：没有内存保护，任何进程都可以读写任何内存地址；没有权限分离，root用户拥有不受限制的权力。随着网络和多用户环境的发展，安全性从“可选功能”变为“必要基础”，导致了Multics的环保护、SELinux的强制访问控制、以及现代微内核（如seL4）的形式化验证[?]。第五代Agent框架正在经历同样的转折：安全性不再只是“加个沙箱”，而是需要从体系结构层面重新设计Agent的权限模型、审计机制和失败恢复策略。

从定律III看五代演进

从第 ??节提出的Agent加速比定律（定律III）的视角看，五代演进对应着 F （可分解比例）和 E （编排效率）的逐步提升。

回顾定律III的形式：

$$S_{\text{agent}} = \frac{1}{(1-F) + \frac{F}{N \cdot E}} \quad (14)$$

其中 $F \in [0, 1]$ 是任务中可以被并行分解的比例， N 是可调度的Agent数量， $E \in (0, 1]$ 是编排效率（1表示无开销，0表示开销吞噬全部并行收益）。

具体数值分析 第一代（ReAct）： $F \approx 0$ （任务不分解，单Agent串行执行）， $N = 1$ ， $E \approx 1$ （无编排开销）， $S_{\text{agent}} \approx 1$ 。这对应“单处理器无并行”的状态。第二代（AutoGPT/Voyager）： $F \approx 0.1-0.2$ （少量任务可以被分离）， $N = 1$ （仍然是单Agent）， $E \approx 0.9$ ， $S_{\text{agent}} \approx 1$ 。分解能力初现，但编排开销不大，因为只有一个Agent。第三代（AIOS/Agents SDK）： $F \approx 0.3$ （中等分解能力）， $N \approx 2-4$ （开始支持多Agent）， $E \approx 0.7$ （调度和上下文切换引入显著开销）， $S_{\text{agent}} \approx$

$1/(0.7 + 0.3/2.8) \approx 1.27$ 。多Agent开始带来加速，但编排开销抵消了部分收益。第四代（Codex/Claude Code）： $F \approx 0.5$ （面向真实软件工程的代码任务有较高的可分解性）， $N \approx 4-8$ （子Agent并发）， $E \approx 0.5$ （沙箱隔离、审批流程、状态同步带来显著开销）， $S_{\text{agent}} \approx 1/(0.5 + 0.5/4) \approx 1.6$ 。加速比显著提升，但编排开销仍是瓶颈。第四代系统的执行时间跨度也正在从分钟级向小时级扩展：Rakuten的工程师测试Claude Code在vLLM项目（1250万行代码）上自主运行7小时完成激活向量提取，达到99.9%数值精度[?]。这种长期自主执行引入了新的体系结构挑战——跨小时的KV缓存持久化、检查点/恢复协议和资源管理——正是Agent运行时类比操作系统的有力支撑。

真正的“模型原生操作系统”需要 $F > 0.8$ 且 $E > 0.8$ ，此时 $S_{\text{agent}} > 1/(0.2 + 0.8/8) \approx 3.3$ ，这意味着既要有好的任务分解（高 F ），又要有低的编排开销（高 E ）。

GAIA[?]、 τ -bench[?]和SWE-bench[?]的最新结果提醒我们，当前系统离这个目标还有相当距离。以SWE-bench Verified为例，截至2025年末，最佳Agent系统在该基准上的通过率约为70–75%——这意味着在四分之一的真实软件工程任务中，Agent仍然无法正确分解和执行。在 τ -bench中，Agent在与真实用户进行多轮交互时的表现更加不稳定，因为用户指令的歧义性和环境的动态性进一步降低了 F 和 E 。

9 设计挑战：性能、状态、一致性与安全

Agent框架从第一代演进到第五代的过程中，每次能力提升都暴露了新的系统性问题。本节将每个设计挑战按四个步骤展开：（1）用具体场景描述问题是什么；（2）分析根本原因；（3）梳理当前的解决方案；（4）与经典计算机中类似问题进行对比。

9.1 延迟、吞吐与成本的三元约束

场景 假设一个代码Agent正在帮助开发者修复一个跨10个文件的bug。Agent需要阅读多个文件（每次阅读触发模型推理）、理解代码结构、定位问题、修改代码、运行测试、查看结果、再修改。整个流程可能涉及20–50次模型调用，每次调用的延迟直接影响开发者的等待时间。如果单次调用延迟是2秒，总延迟可能超过1分钟。如果服务提供商为了降低延迟而增加GPU并行度，每次调用的成本会显著上升。这就是“延迟–吞吐–成本”三元约束：降低延迟需要更多资源（增加成本），提高吞吐需要更大的批处理（增加延迟），控制成本需要更少的资源（同时增加延迟和降低吞吐）。

根本原因 大模型推理的解码阶段存在一个经典的结构矛盾。预填充（prefill）阶段是**计算密集型的**：需要处理整个输入序列，GPU的计算单元满负荷工作，瓶颈在FLOPS。解码（decode）阶段是**内存带宽密集型的**：每生成一个新token，都需要读取全部模型权重和当前KV缓存——这本质上是“每个时钟周期都要从内存搬移一遍参数”[?]。Gholami等人量化了这一结构性失衡：峰值FLOPS以每两年3.0倍的速度增长，而DRAM带宽仅以每两年1.6倍的速度增长[?]。

这意味着两个阶段对硬件资源的需求完全不同，将它们混合在同一个批处理中必然导致资源利用不均：预填充阶段抢占了计算资源，解码阶段等待内存带宽。此外，Agent场景下的请求模式更加复杂：多个子Agent并发发起请求，请求之间的上下文长度差异巨大（有的只读一个函数，有的需要理解整个仓库），这使得传统的静态批处理策略效率低下。

当前解决方案 研究社区从多个层面应对这一约束：

（1）**调度层优化** ORCA[?]提出迭代级调度（iteration-level scheduling）：不等待整个序列生成完毕再调度新请求，而是每个解码步结束后检查是否可以加入新请求，显著提高了GPU利用率。DistServe[?]将预填充和解码彻底分离到不同的GPU组，使每组可以独立优化：预填充组用高FLOPS的GPU，解码组用高内存带宽的GPU。Sarathi-Serve[?]引入了stall-free scheduling：通过chunked prefill将长输入切分为固定大小的块，与解码任务混合调度，消除了解码阶段因等待预填充而产生的停顿。

(2) **内存管理优化** vLLM[?, ?]通过PagedAttention将KV缓存组织为固定大小的页，通过block table映射到非连续物理空间，实现了KV缓存的按需分配和共享。SGLang[?, ?]通过RadixAttention将请求前缀组织为基数树（radix tree），自动识别和复用共享前缀的KV缓存。TGI[?]和TensorRT-LLM[?]则在工业级部署中实现了连续批处理和页式KV管理的工程优化。vAttention[?]提供了另一种思路：利用OS级别的连续虚拟内存和动态增长，直接支持标准注意力内核，避免了对PagedAttention专用内核的依赖。

与经典计算机的类比 这组问题与经典计算机体系结构中的“内存墙”（Memory Wall）问题高度类似。1980年代以来，CPU速度以每年约50%的速度增长，而DRAM延迟仅以每年约7%的速度改善[?]。体系结构的应对是多级缓存（L1/L2/L3）、预取（prefetching）和写缓冲（write buffer）—对应LLM推理中的KV缓存分层、前缀缓存和异步解码。分时OS中的“响应时间-吞吐量”权衡也与此对应：交互式任务需要低响应时间（低延迟），批处理任务需要高吞吐量（大batch），调度器需要在两者之间寻找平衡点[?]

9.2 状态管理与跨会话一致性

场景 假设一个用户连续三天使用同一个Agent来开发一个项目。第一天，Agent学会了用户偏好“使用TypeScript而不是JavaScript”。第二天，用户要求Agent重构代码，Agent需要在遵循第一天学到的偏好的同时，正确理解代码的当前状态。第三天，用户发现了一个新bug，Agent需要回忆前两天做过哪些修改（因为bug可能是前两天引入的），同时理解项目的最新状态。更进一步，如果用户在第二天同时运行了两个Agent实例（一个做重构，一个写测试），两个Agent可能同时修改同一个文件，产生冲突。

根本原因 传统CPU是无状态执行器：给定相同的输入和寄存器状态，输出是确定且可重复的。而智能系统的核心挑战是必须跨会话**持久化状态**—包括用户偏好（“我喜欢TypeScript”）、历史任务记录（“昨天我做了重构”）、以及外部世界变化（“代码仓库新增三个文件”）。一旦引入持久状态，至少三类问题必然出现：

(1) **多Agent共享记忆的并发问题**：当多个Agent同时读写共享记忆时，如何保证一致性？这类似于多线程编程中的竞态条件（race condition）。

(2) **延迟提交与冲突合并**：Agent A修改了文件，但尚未提交；Agent B基于旧版本开始修改同一文件。如何检测和解决冲突？这类似于分布式系统中的乐观并发控制（optimistic concurrency control）。

(3) **记忆时效性**：三天前学到的“项目使用React”可能已经过时（项目迁移到了Vue）。如何判断记忆是否仍然有效？这类似于缓存一致性中的失效（invalidation）问题。

当前解决方案 LongMemEval[?]的评估说明，长期记忆并不是简单地“存更多聊天记录”，而是涉及四种能力：信息抽取（从对话中提取关键事实）、时序推理（理解事件的先后顺序）、知识更新（当新信息与旧记忆冲突时更新记忆）、和拒答能力（当记忆不足以支持回答时主动拒绝）。

MemGPT[?]借鉴了OS虚拟内存的分页和换入换出机制来管理上下文，将上下文分为“主存”（当前活跃的对话窗口）和“外存”（向量数据库中的长期记忆），通过编辑和检索操作在两者之间移动信息。MemoryOS[?]将记忆管理抽象为操作系统的内存管理模块，实现了结构化的记忆编码、检索和更新。A-MEM[?]提出了Agent式的记忆管理：记忆不再是被动存储的字符串，而是由Agent自主决定何时写入、何时更新、何时删除的“活跃实体”。Letta[?]（前身为MemGPT项目）进一步将“有状态Agent”作为核心抽象，显式区分了对话状态、核心记忆和归档记忆。

从分布式系统角度看，至少需要区分三种状态类型[?, ?, ?]：(1) **局部可丢弃状态**（ephemeral state）：当前推理步骤的中间结果，类似于CPU寄存器，不需要持久化。(2) **需因果有序的会话状态**（session state）：同一任务内Agent间的消息传递和状态传递，类似于分布式系统中的因果一致性强保证了操作间的先后顺序[?](3) **需要强可追溯和可回放的提交状态**（committed state）：对文件、数据库或外部系统的永久修改，类似于数据库事务的commit point，需要支持审计和回滚[?]

值得注意的是，Agent执行的时间跨度正在从分钟级向小时甚至天级扩展。最新的工业实践表明，Agent已能自主运行7小时以上完成复杂任务[?]。这种跨小时乃至跨天的持续执行引入了额外的时序衰减挑战： $\beta(L)$ 的衰减不仅与上下文位置有关，还与会话持续时间和跨步骤的状态积累有关。这意味着上述三种状态类型的划分需要增加一个时间维度：跨小时执行需要KV缓存的会话持久化（L2），上下文窗口管理需要支持跨检查点的工作集恢复（L3），治理层需要定义定期检查点与人工恢复协议（L5）。定律II当前的模型尚未捕捉这一时序维度，是后续研究的重要方向。

与经典计算机的类比 这组问题对应经典计算机中的**虚拟内存与分布式一致性问题**。OS的虚拟内存通过页表将虚拟地址映射到物理地址，并提供换入换出机制来制造“内存无限大”的幻觉—对应Agent系统的上下文管理。分布式文件系统（如GFS/Spanner[?]）通过Paxos/Raft[?]共识协议保证多副本的一致性—对应多Agent共享记忆的一致性。CAP定理[?]指出一致性、可用性和分区容错性三者不可兼得—Agent系统的记忆管理同样面临这一权衡：是选择强一致性（每次操作都等待所有副本同步）还是最终一致性（允许短暂的不一致以换取低延迟）。

9.3 接口漂移与版本兼容

场景 一个Agent被配置为使用GitHub API来管理代码仓库。某天GitHub更新了API，将`repos/:owner/:repo/pulls`的返回格式从v3改为v4，新增了分页参数，删除了部分字段。Agent的工具调用开始返回错误或解析失败。更复杂的情况：Agent使用的提示模板依赖模型的特定输出格式（如“always respond in JSON”），但模型升级后输出风格发生了微妙变化，导致下游解析器频繁崩溃。类似的问题还出现在MCP服务器的能力描述更新、技能包的接口变更、以及记忆文件格式的版本升级中。

根本原因 传统计算机系统之所以可扩展，很大程度上源于ISA、ABI、系统调用接口与文件格式的**长期稳定性**。x86指令集从1978年的8086到2026年的最新处理器，核心指令集保持了向后兼容；POSIX标准自1988年确立以来，`read()/write()/fork()`等系统调用的语义几乎没有变化[?]。这种稳定性使得上层软件可以安心构建，而不必担心底层接口突然变化。

相比之下，当前Agent生态存在严重的**接口漂移**（interface drift）问题。漂移的来源至少有四种：（1）**模型版本漂移**：模型升级后，对相同提示的输出格式可能变化；（2）**工具schema漂移**：外部API的接口描述和返回格式可能随时更新[?]；（3）**协议漂移**：MCP、A2A等协议本身还在快速迭代[?, ?]；（4）**技能包漂移**：可复用的Agent技能在不同项目或不同模型版本上的表现可能不同。

当前解决方案 OpenAI Codex通过AGENTS.md[?]和Skills[?]机制提供了项目级的接口稳定锚点：技能包有明确的版本号和兼容性声明。Claude Code通过CLAUDE.md[?]实现类似的功能。Agent互操作性协议（MCP[?]、A2A[?]）正在尝试建立Agent与工具之间、Agent与Agent之间的标准化接口。A2A协议特别关注Agent间通信的标准化：它基于JSON-RPC 2.0 over HTTP，定义了Agent发现、能力协商和任务委派的标准流程[?]。然而，这些协议本身也在快速迭代，尚未达到ISA或POSIX那样的稳定性。

与经典计算机的类比 这对应经典计算机中的**ABI稳定性与版本管理**问题。Intel维持x86 ISA的向后兼容已经超过45年；Linux内核维持系统调用的向后兼容已经超过30年；甚至共享库的版本管理也有成熟的方案（如semantic versioning和symbol versioning）。模型原生计算架构需要建立自己的“版本控制学”：工具schema需要版本号，模型输出格式需要兼容性声明，技能包需要依赖管理（类似npm的package.json），记忆文件需要格式迁移工具（类似数据库的schema migration）。

9.4 安全、隐私与最小权限

场景 一个代码Agent正在帮助开发者处理用户数据。Agent需要读取包含用户隐私的数据库表结构（以理解数据模型）、修改处理用户数据的代码、运行测试（测试数据中可能包含真实用

户信息)。在这个过程中, Agent可能: (1) 将敏感数据写入日志文件或发送到远程API (数据泄露); (2) 被恶意的代码注释或文件内容“提示注入”, 执行非预期的操作 (目标劫持); (3) 在测试中使用真实用户数据而非模拟数据 (隐私违规)。更危险的情况: Agent可以运行任意shell命令, 一个精心构造的提示注入可能让Agent执行`rm -rf /`或上传敏感文件到外部服务器。

根本原因 一旦Agent可以读写文件、运行命令、浏览网页、调用数据库, 它就不再只是语言模型, 而是具备外部行动能力的**主体** (agent in the security sense)。这带来了传统软件安全无法覆盖的新攻击面:

(1) **提示注入** (prompt injection): 攻击者通过在工具返回的数据、文件内容、甚至网页中嵌入恶意指令, 劫持Agent的行为。与传统的代码注入 (如SQL注入) 不同, 提示注入利用的不是代码逻辑的缺陷, 而是模型对自然语言的“误解”—模型无法区分“用户的指令”和“数据中嵌入的指令”[?, ?]。

(2) **权限过度** (privilege overprovisioning): 当前大多数Agent在获得工具访问权限时, 获得的是该工具的全量权限 (如“可以读写整个文件系统”), 而不是最小必要权限 (如“只能读写项目目录下的特定文件”)。

(3) **记忆污染传播** (memory taint propagation): 攻击者通过一次成功的提示注入, 将恶意信息写入Agent的长期记忆, 这些“被污染”的记忆会在后续所有会话中影响Agent的行为, 形成持久的后门[?]。

当前解决方案 工业界和学术界正在从多个维度构建防御体系:

(1) **沙箱隔离**。Codex默认在OS强制沙箱内运行, 每个任务在独立的容器中执行, 无法访问宿主机的文件系统或网络[?]. Claude Code默认只读, 任何写操作都需要用户显式授权[?]. Aura在移动端为每个App Agent建立独立的沙箱, 并限制Agent Kernel对系统资源的访问[?]。

(2) **能力安全模型**。CaMeL[?]用操作系统的capability安全原则防御提示注入: Agent不能直接执行有副作用的操作, 而是通过一个确定性的capability manager获取“临时通行证”。每个通行证只授权一个特定操作 (如“只读文件`src/main.py`”), 用后即废。这从根本上消除了“提示注入导致权限提升”的可能, 因为权限是由确定性代码而非概率式模型决定的。

(3) **审批与审计**。Codex的approval policies[?]允许开发者定义哪些操作需要人工审批 (如“任何DELETE操作都需要确认”)。Agents SDK的tracing机制[?]记录了每一次工具调用的完整上下文, 支持事后审计和回放。

(4) **安全形式化**。IronClaw[?]提出了面向Agent安全的形式化框架, 将Agent的行为空间和权限空间显式建模, 通过静态分析和运行时监控来检测越界行为。Christodorescu等人[?]进一步论证了Agent安全需要借鉴操作系统安全的全部经验: 最小权限、隔离、审计、失败安全 (fail-safe)。

与经典计算机的类比 这对应经典计算机中的**进程安全与权限管理**。Unix的安全模型建立在三个核心概念上: (1) 用户态/内核态分离—用户程序不能直接访问硬件, 必须通过系统调用[?]; (2) 文件权限 (rwx)—每个文件有明确的读写执行权限; (3) root/非root分离—特权操作需要权限提升。Agent安全需要类似的三层模型: (1) 概率平面/确定性平面分离—Agent不能直接执行有副作用的操作, 必须通过确定性平面的审批通道; (2) 工具capability标注—每个工具有明确的能力声明和权限要求; (3) 治理层权限提升—高风险操作需要显式授权或人工确认。

双重用途的安全挑战 值得警惕的是, Agent能力的双重用途特性引入了对抗性升级的动态。同样的Agent能力既可以用于自动化安全审查 (防御), 也可以用于自动化漏洞发现和利用 (攻击) [?]. 这意味着双平面架构的确定性控制平面不仅要约束自身Agent的行为, 还必须预判对手也拥有等效的双平面系统。报告预测“Agent化网络防御系统”将以机器速度运行[?], 这暗示L5治理层的原语设计需要将对抗性场景纳入安全威胁模型。

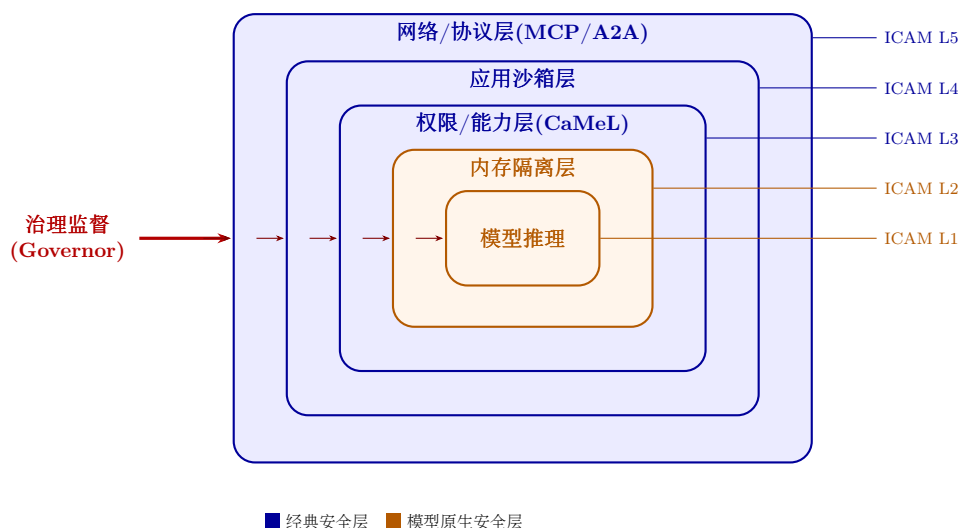


Figure 9: 基于ICAM分层的安全防护架构

9.5 治理层：体系结构的第一性重点

问题的核心 ArbiterOS[?]指出，Agent工程的根本危机来自一个结构性矛盾：开发者习惯用**确定性软件思维**来驱动一个**概率式处理器**。传统软件工程中，代码的行为是确定性的：相同的输入必然产生相同的输出，测试可以覆盖所有分支，bug可以通过断言来检测。但Agent的行为由概率式模型驱动：相同的提示可能产生不同的输出，测试无法覆盖所有可能的推理路径，“bug”可能以模型产生幻觉或不遵循指令的形式出现，且难以通过传统的断言来检测。

Aura[?]进一步说明，一旦Agent进入操作系统和移动端环境，权限管理（Agent是否有权访问联系人列表？）、身份验证（调用Agent的是真正的用户还是攻击者？）、语义输入污染（一条恶意短信是否可以劫持Agent？）、以及记忆taint传播（被污染的记忆如何清理？）都会变成**内核级问题**——不再是某个应用的安全漏洞，而是整个系统体系结构的安全基础。

图 ??展示了基于ICAM分层的安全防护架构。

治理不应是事后的补丁 这说明，治理（governance）不应被当作事后的安全补丁或合规检查清单，而应被设计为体系结构的**第一性重点**。正如操作系统设计者不会先写完所有功能再加安全模块（这种做法已经被历史证明不可行[?]），Agent系统的治理也必须从第一天就融入体系结构设计。

ICAM各层的治理机制 具体而言，ICAM（定义见第 ??节）的每一层都需要对应的治理机制：

- **L1 物理执行层的治理**：硬件级的推理精度保证和能耗预算。类似于CPU的温度监控和频率调节（thermal throttling），L1需要确保推理精度不低于安全阈值（如量化不能导致关键判断出错），以及GPU/TPU的能耗在可控范围内。
- **L2 推理服务层的治理**：推理预算（inference budget）和精度治理。每个Agent的推理调用应该有预算上限（如“每个子任务最多10次推理调用”），超出预算应触发失败恢复而非无限重试。KV缓存管理需要保证关键状态不被意外驱逐。
- **L3 上下文管理层的治理**：记忆保留和删除策略（data retention policy）。用户的敏感信息（如密码、信用卡号）应有自动检测和及时删除机制（类似于GDPR的“被遗忘权”）。记忆的时效性应有明确的TTL（time-to-live）标注，过期记忆应自动降级或删除。
- **L4 语义接口层的治理**：工具能力标注（tool capability annotation）和调用审计（tool call audit）。每个外部工具应有标准化的能力声明（“这个工具能做什么、不能做什么、有什么副作用”），每次工具调用都应被记录在可审计的trace中。MCP协议[?]正在朝这个方向发展。
- **L5 编排层的治理**：权限审批（permission approval）和失败恢复（failure recovery）。

高风险操作（写入文件、发送邮件、执行shell命令）应通过确定性平面的审批流程。失败恢复应有明确的回滚策略（如git revert）和人工接管机制。

- **L6 应用层的治理**：用户意图确认（intent confirmation）和副作用声明（side-effect declaration）。在执行任何不可逆操作前，系统应向用户确认理解正确。每个用户任务执行完毕后，系统应生成副作用报告（“本次任务修改了哪些文件、发送了哪些邮件、访问了哪些外部服务”）。

双平面架构中的治理定位 从第 ??节提出的双平面架构视角看，治理层正是**确定性控制平面**中的核心组件。它的职责不是替代概率平面的推理能力，而是为推理结果设置边界、审计轨迹和回滚机制。具体来说：概率平面负责“理解用户意图、分解任务、生成工具调用参数”—这是LLM擅长的推理能力；确定性平面负责“检查权限是否满足、工具调用是否安全、结果是否合规、失败是否可恢复”—这是传统软件工程擅长的确定性逻辑。CaMeL[?]的capability manager、Codex[?]的approval policies、Claude Code[?]的hooks都是这一原则的具体实现。

这与传统操作系统中“内核不替用户程序做计算，但保证计算不会越界”的设计哲学完全一致[?]：内核不会帮你排序数组，但保证你的程序不会读写其他进程的内存。同样，治理层不会替Agent做推理，但保证Agent的推理结果不会导致越权操作、数据泄露或不可恢复的失败。

治理的形式化挑战 将治理融入体系结构还面临一个根本性的形式化挑战：如何为概率式行为定义“安全边界”？传统OS的安全模型建立在确定性状态机上：每个系统调用都有明确的前置条件（如“文件描述符必须有效”）和后置条件（如“返回读取的字节数”）。但Agent的行为是概率式的：同一个操作在不同上下文中可能产生不同结果，且结果的好坏（“是否安全”）往往依赖语义判断而非形式化验证。ArbiterOS[?]提出的“策略规范”（policy specification）和IronClaw[?]提出的行为空间建模是朝这个方向迈出的第一步，但距实用的形式化治理框架还有相当距离。这恰恰是ICAM和双平面架构为未来研究划出的关键方向之一。

10 开源实现与工程实践

从工程实现看，当前生态已经出现了若干可作为“未来模型原生计算架构原型”的项目。本节按照ICAM的层次结构，从服务层（L2）、Agent层（L5）和协议层（L4）三个维度，逐一介绍代表性系统的核心特点及其在ICAM中的定位。

10.1 服务层（L2）：推理服务与KV缓存管理

推理服务层对应ICAM的L2层，负责模型权重加载、KV缓存管理、连续批处理和预填充/解码调度。目前这一层已形成较为成熟的工程生态。

vLLM以PagedAttention为核心，将KV缓存组织为固定大小的页（block），通过block table映射到非连续物理显存空间，实现了类似操作系统虚拟内存的显存管理[?, ?]。其自动前缀缓存（Automatic Prefix Caching）功能进一步支持了跨请求的共享前缀复用，相当于L2缓存中的共享只读代码页[?]

SGLang强调结构化程序执行与RadixAttention[?, ?]。RadixAttention用基数树（radix tree）索引所有历史请求的token前缀，自动发现和复用最大公共前缀的KV缓存，在多轮对话和批量推理场景下展现出更高的缓存命中率。实测表明，SGLang在某些工作负载下吞吐量比vLLM高出约29%[?]

TGI（Text Generation Inference）与**TensorRT-LLM**分别由Hugging Face和NVIDIA提供，展示了面向生产环境的部署能力[?, ?]。前者侧重多模型统一接口和易用性，后者利用NVIDIA GPU的深度优化（如FP8量化、Tensor Core加速）追求极致吞吐。这三类系统（vLLM、SGLang、TGI/TensorRT-LLM）分别代表了“类虚拟内存管理”“结构化缓存复用”和“硬件深度优化”三种工程路线，共同构成了L2层的实现谱系。

10.2 Agent层（L5）：Agent运行时与操作系统

Agent层对应ICAM的L5层，负责任务规划、Agent调度、权限审批、失败恢复和状态提交。这一层的开源生态最为活跃，方向也最为多样。

在通用Agent方向，**OpenHands**提供了一个开放的Agent平台，支持多种LLM后端、沙箱执行和可扩展的工具接口[?]; **SWE-agent**专注于软件工程任务，通过Agent-Computer Interface (ACI) 将代码编辑、测试和调试统一为结构化操作[?]; **AutoGPT**从自主持续工作流出发，探索了Agent的自动目标分解和长期执行能力[?]; **Letta**（原MemGPT团队）则聚焦于"有状态Agent"，将长期记忆作为Agent的一等公民来管理[?]

在Agent OS方向，**ArbiterOS**引入了治理层概念，提出"governor治理概率CPU"的分层架构[?]; **AgentOS**提出了受结构化操作系统逻辑约束的"reasoning kernel"[?]; **UFO²**展示了桌面级AgentOS的可行性，将HostAgent、AppAgent和GUI-API操作层组织为层次结构[?]; **Aura**则在移动端探索了安全优先的Agent架构，包含System Agent、沙箱化的App Agents和Agent Kernel[?]

在工业系统方向，**OpenAI Codex**明确将子Agent (subagents)、技能 (skills)、MCP服务器接入、沙箱执行和审批策略 (approval policies) 作为一等公民[?, ?, ?, ?, ?]; **Anthropic Claude Code**提供子Agent、技能、hooks、MCP连接和项目记忆文件，并默认采用只读权限与审批机制[?, ?]。这两个系统说明，工业界已经开始把Agent运行时当作操作系统来构建。

10.3 协议层（L4）：标准化工具连接与Agent互联

协议层对应ICAM的L4层，负责统一外部能力的访问接口。这一层正在从"点对点工具调用"向"标准化总线协议"转变。

MCP (Model Context Protocol) 提供了标准化的工具连接协议[?]。截至2025年底，Anthropic报告已有超过10,000个活跃的公开MCP服务器[?]。MCP将连接AI应用与外部数据源、工具、工作流统一为JSON-RPC规范，扮演类似PCIe总线或USB在传统计算机中的角色。

A2A (Agent-to-Agent Protocol) 由Google于2025年4月推出，定位为Agent间通信标准[?, ?]。它支持能力发现 (capability discovery)、任务生命周期管理和多模态协同，已被捐赠给Linux基金会进行中立治理[?]。A2A扮演类似网络协议栈中TCP/IP的角色，与MCP互补：MCP是Agent到工具的纵向总线，A2A是Agent到Agent的横向互联。

10.4 五条实现建议

基于上述工程实践和本文提出的六条设计公理，本文给出五条可直接指导工程落地的实现建议：

第一，控制面与数据面分离 将确定性控制逻辑（权限审批、日志审计、失败恢复）从概率式推理路径中剥离，分别对应双平面架构中的确定性控制平面和概率执行平面。例如，Codex的approval policies和Claude Code的hooks就是这一分离的具体实现[?, ?]。

第二，短期上下文与长期记忆分离 将高频使用的"热上下文"（当前对话窗口）与低频访问的"冷记忆"（历史摘要、知识库索引）分到不同的存储介质和管理策略中。这对应ICAM L3层的冷热分层设计，MemGPT的虚拟上下文管理就是这一原则的典型实践[?]。Augment Code的企业案例——CTO预估4-8个月的项目在2周内完成[?]——展示了有效的上下文管理对加速开发者入职和项目交付的实际影响。

第三，所有可影响行为的对象都要版本化 提示模板、工具schema、技能包、记忆文件、MCP服务器能力描述都应附带版本号和变更日志[?, ?]。传统计算机通过ISA和ABI的稳定性保证向后兼容；模型原生计算系统同样需要这种"接口契约"来应对快速演化。

第四，像OS一样实施资源配额 对每个Agent或任务设定显式的资源预算：上下文窗口上限、工具调用次数上限、推理时间上限和成本上限[?]。这对应公理5（最小权限公理）和公

理6（可观测性公理）在资源管理层面的具体化。

第五，像分布式系统一样设计故障恢复 Agent执行的长任务可能因模型超时、工具失败或上下文溢出而中断。系统应像分布式数据库一样提供检查点（checkpoint）、重试（retry）和回滚（rollback）机制，确保副作用操作可审计、可撤销[?, ?]。Rakuten的Claude Code自主完成了vLLM项目中1250万行代码上的激活向量提取任务（7小时，99.9%数值精度）[?]，表明具备隐式检查点和重试机制的长时运行Agent已经能够处理工业级任务。

11 研究路线图

表 ??给出一个基于ICAM六层模型的研究路线图，按短期（1–2年）、中期（2–4年）和长期（4–8年）三个阶段组织。每个议题明确说明（1）要解决什么问题、（2）用什么方法、（3）期望的结果。

Table 5: 基于ICAM的研究议题清单

阶段	ICAM层	议题	方法	度量指标	相关定律/公理
短期	L2	KV缓存命中率优化	语义驱逐策略、KV量化压缩、前缀共享	H, S , TTFT	定律I
短期	L3	统一上下文编译器	热/温/冷分层管理，版本戳与TTL标注	W_{eff} , 压缩比	定律II
短期	L4	Tool ABI标准化	Schema语义版本化、Capability标注	成功率、安全违规率	公理5
短期	L5	软件工程Agent内核	ACI接口统一，子Agent专业化分工	任务完成率、 E	定律III
中期	L3	语义页置换策略	学习型Eviction策略、摘要回写检索	$\beta(L)$ 曲线	定律II
中期	L5	一致性可控共享记忆	事件日志（Event Sourcing）、冲突检测	记忆正确率	公理4
中期	L2	异构模型协同	草稿模型（Speculative Decoding）、MoE路由	端到端时延、 H	定律I
中期	L4	Agent安全架构	沙箱隔离、审计日志、能力安全（Capability）	审计完整性	公理5,6
长期	L1–L6	智能ISA与可编程技能层	任务DSL、受控输出格式、可复用技能包	泛化能力	公理2,3
长期	L3–L5	状态化个体Agent	长期记忆、经验抽象与策略学习	经验利用率	定律II,III
长期	L1–L6	分布式模型原生计算组织	记忆复制、一致性层、负载均衡	集群利用率	定律I,III
长期	L5	治理层原语	概率/确定性双平面、审计日志、权限声明	治理覆盖率	公理3,5,6

11.1 短期议题

短期议题聚焦于已有工程原型的优化和标准化，目标是让现有系统的性能和可靠性达到生产级别。

KV缓存命中率优化（L2） 当前问题：大模型推理的解码阶段是典型的内存带宽受限问题[?]，KV缓存的命中率 H 直接决定了推理加速比 S 。方法：通过语义驱逐策略（如H2O的重击者识别[?]）、KV量化压缩（如KVQuant的8倍压缩[?]、KIVI的非对称2-bit量化[?]）和前缀共享（如PagedAttention[?]、RadixAttention[?]）来提升 H 。期望结果：在典型Agent工作负载下将 H 从当前的0.5–0.7提升至0.85以上，使定律I预测的加速比 S 达到5–10倍。

统一上下文编译器 (L3) 当前问题: 不同Agent框架各有自己的上下文管理策略 (MemGPT的虚拟上下文、Letta的有状态Agent、HiAgent的层次化工作记忆[?, ?, ?]), 缺少统一的"上下文编译器"来决定哪些信息以原文加载、哪些以摘要加载、哪些只保留索引。方法: 借鉴操作系统的页置换和编译优化, 设计热/温/冷三层的上下文管理策略, 并为每层附带的上下文块加上版本戳 (version stamp) 和时效标注 (TTL)。期望结果: 在保持任务完成率不降的前提下, 将有效上下文利用率 W_{eff}/C 提升至少50%, 验证定律II的预测。

Tool ABI标准化 (L4) 当前问题: 不同Agent框架的工具调用接口各异, MCP和A2A提供了协议层面的标准化, 但工具schema本身仍缺少版本化、向后兼容和capability标注的统一规范[?, ?]。方法: 为工具schema引入语义版本号 (如MCP的schema versioning)、输入输出类型约束、副作用声明和capability标注, 使L4层真正成为可编程的"应用二进制接口" (ABI)。期望结果: 工具调用成功率提升, 安全违规率下降, 为公理5 (最小权限) 的落地提供接口基础。

软件工程Agent内核 (L5) 当前问题: SWE-bench和SWE-agent表明, 当前代码Agent在真实GitHub问题上的解决率仍然有限[?, ?]。Agent-Computer Interface (ACI) 的设计缺乏统一标准, 不同框架的子Agent分工策略各异。方法: 统一ACI规范, 设计专门的子Agent分工策略 (如代码搜索Agent、编辑Agent、测试Agent), 并通过定律III的 E (编排效率) 来量化评估。期望结果: 在SWE-bench等基准上将任务完成率提升, 同时将 E 从当前的约0.3–0.5提升至0.6以上。

11.2 中期议题

中期议题聚焦于跨层协同和系统级安全, 目标是让模型原生计算系统从"单点优化"走向"全栈协同"。

语义页置换策略 (L3) 当前问题: 传统操作系统的页置换 (如LRU、CLOCK) 基于确定性地址访问模式, 而智能系统的"页置换"需要基于语义相关性来决定驱逐哪些上下文块。方法: 设计学习型eviction策略, 根据上下文块与当前任务的语义相关度来决定保留或驱逐, 对被驱逐的块生成摘要以便回写检索。期望结果: 绘制出 $\beta(L)$ 曲线的精确形状, 为定律II提供定量参数。

一致性可控共享记忆 (L5) 当前问题: 多Agent协作时共享记忆的一致性管理缺乏统一框架。不同Agent可能同时修改同一记忆块, 导致冲突和信息丢失。方法: 引入事件日志 (event sourcing) 和冲突检测机制, 设计可调节一致性级别 (从最终一致到强一致) 的记忆访问协议[?, ?]。期望结果: 在多Agent基准上实现记忆正确率不低于单Agent基线, 为公理4 (虚拟化公理) 在记忆层面的落地提供实证。

异构模型协同 (L2) 当前问题: 不同规模的模型在延迟、成本和能力上各有优劣, 但当前系统通常只使用单一模型。方法: 通过草稿模型 (speculative decoding[?]) 和MoE路由[?, ?]实现异构模型协同, 让简单任务用小模型、复杂任务路由到大模型。期望结果: 在保持输出质量的前提下降低端到端时延, 同时提升KV缓存命中率 H 。

Agent安全架构 (L4) 当前问题: 随着Agent获得越来越多的外部行动能力 (读写文件、运行命令、联网), 安全边界变得模糊。方法: 为Agent运行时内建沙箱、审计和能力安全机制[?, ?, ?], 确保每个有副作用的操作都经过显式审批并产生可审计的事件记录。期望结果: 实现审计完整性100% (所有副作用操作均可追溯), 为公理5和公理6的落地提供安全基础设施。

11.3 长期议题

长期议题聚焦于架构层面的根本性创新, 目标是构建真正的模型原生计算体系结构。

智能ISA与可编程技能层 (L1–L6) 当前问题: 提示词、工具描述和技能包构成了模型原生计算的"软指令", 但它们缺少形式化语义和稳定性。方法: 发展任务DSL (Domain-Specific Language)、受控输出格式和可复用技能包, 建立类似传统ISA的稳定接口层。期望结果: Agent的泛化能力显著提升, 使得同一技能包可以在不同模型和运行时之间无缝迁移。

状态化个体Agent (L3–L5) 当前问题：大多数Agent是无状态的——每次会话从零开始，无法积累和复用经验。方法：为Agent引入长期记忆和经验抽象机制，使其能从历史执行中学习策略、总结模式并主动优化行为[?, ?]。期望结果：经验利用率（历史经验对当前任务的贡献比例）显著提升，为定律II和定律III提供新的验证维度。

分布式模型原生计算织构 (L1–L6) 当前问题：当多个Agent集群跨节点协作时，记忆复制、状态一致性和负载均衡等分布式系统问题尚未被系统化地解决。方法：借鉴分布式数据库的记忆复制和一致性层设计[?, ?]，为多节点Agent集群提供统一的资源调度和状态管理。期望结果：集群利用率随节点数线性增长（或至少亚线性增长），验证定律I和定律III在分布式场景下的适用性。

治理层原语 (L5) 当前问题：现有Agent系统的治理机制（权限、审计、合规）大多是事后添加的补丁，而非体系结构的一等公民。方法：将概率/确定性双平面、审计日志和权限声明设计为体系结构的核心原语[?, ?]，确保每一层都有对应的治理机制。期望结果：治理覆盖率100%（每一层的每个有副作用的操作都受到治理约束），实现公理3、5、6的全栈落地。

值得强调的是，这些议题并不要求新模型先行。许多问题完全可以在现有模型之上开展研究——例如上下文编译器、Tool ABI标准化和Agent安全架构都是系统层面的工程问题，与基础模型的能力提升解耦。这传统计算机史相似：架构突破往往不只来自晶体管更强，也来自接口和运行时更成熟。ISA的稳定性让软件生态繁荣，虚拟内存的发明让程序不再受物理内存限制，POSIX的标准化让应用可以跨平台迁移。模型原生计算系统同样需要这些"系统层面的突破"。

12 伦理、安全与可解释性

把大模型系统视为计算机，并不意味着伦理问题会自动消失；相反，体系结构的视角让某些伦理问题更加清晰——因为它为每个问题指定了具体的ICAM层次和对应的治理机制。

12.1 权限与隔离（对应L5编排层）

如果Agent相当于进程，那么它拥有哪些权限、能看到哪些秘密、是否能联网、是否能修改持久状态，就必须像操作系统安全策略那样被显式配置[?, ?, ?, ?, ?, ?]。

具体而言，ICAM L5层的治理机制需要回答以下问题：

- **谁能创建Agent?** ——对应进程创建权限。Codex要求用户通过AGENTS.md显式声明Agent行为边界[?]
- **Agent能访问哪些资源?** ——对应文件系统权限。Claude Code默认只读，写操作需显式审批[?]
- **Agent的权限如何粒度化?** ——对应capability-based安全。CaMeL用能力令牌（capability token）控制每个工具调用的权限边界[?]
- **Agent越权时如何终止?** ——对应进程信号和沙箱隔离。Codex使用OS级沙箱，Agent的所有写操作都被隔离在受控环境中[?]

从双平面架构的视角看，这些权限控制属于确定性控制平面的核心职责：概率执行平面负责"能做什么"（推理和生成），确定性控制平面负责"允许做什么"（审批和隔离）。

12.2 隐私与记忆治理（对应L3记忆层）

隐私问题集中在L3（记忆层）。一旦引入长期状态，系统必须回答四个核心问题——这些问题在传统计算机的内存管理中也有对应，但在语义系统中变得更加复杂：

什么应保存? 不是所有交互都应被长期记忆。传统操作系统通过页面保护位（protection bits）区分可读、可写、可执行页面；L3记忆层同样需要区分"可持久化"和"仅会话级"的信

息。例如，用户的代码编辑偏好可以持久化，但临时密码或敏感对话内容应标记为"仅会话级"，会话结束后自动清除。

保存多久？ 这涉及记忆的保留策略（retention policy）。传统文件系统通过TTL（Time-To-Live）管理临时文件；L3层同样需要为每个记忆块设定TTL和版本号。更复杂的是，欧盟《人工智能法案》（EU AI Act）要求高风险AI系统保留至少10年的操作日志[?]，而GDPR的"被遗忘权"（Right to Be Forgotten）则要求用户可以请求删除个人数据[?]——两者之间存在直接的政策冲突，需要L3层的保留策略设计者显式处理。

谁有权删除？ 在多Agent共享记忆的场景下，一个Agent写入的记忆块可能被其他Agent引用。删除操作不能简单执行，而需要类似垃圾回收（garbage collection）的引用追踪机制——只有当所有引用者都同意或已失效时，记忆块才可被真正清除。这对应公理6（可观测性公理）的延伸：删除操作本身也必须是可审计的。

摘要能否反推出原始敏感信息？ L3层的上下文编译器经常将原始信息压缩为摘要以节省窗口空间。但摘要可能保留足够多的语义线索，使得敏感信息（如个人身份、财务数据）可以从摘要中推断出来。这要求L3层的语义地址契约包含最小保留（只保留任务必需的信息）、可追溯删除（删除时连同所有派生摘要一起清除）、访问隔离（不同用户的记忆不可交叉访问）和加密存储[?, ?]。

12.3 可解释性：从"解释模型"到"解释系统"（对应全栈）

传统LLM可解释性研究的重点是解释模型的内部表征（如注意力头的作用、神经元的语义）。但ICAM认为，对于一个运行在完整模型原生计算架构上的Agent系统，更有操作性的解释单元不在模型内部，而在系统行为轨迹中——这正是可观测性公理（公理6）的核心。

具体而言，ICAM建议将可解释性分解为以下几个可操作的层级：

- **L2层**——缓存命中/未命中日志：哪些上下文被复用、哪些被驱逐、驱逐策略的依据是什么。
- **L3层**——上下文编译决策：哪些信息以原文加载、哪些以摘要加载、摘要的来源和压缩比是多少。
- **L4层**——工具调用参数与结果：调用了什么工具、传递了什么参数、工具返回了什么结果、是否触发了权限拒绝。
- **L5层**——审批决策与状态提交：哪个Agent在什么时刻请求了什么权限、审批是被批准还是被拒绝、最终提交了什么状态变更。

这种"系统级行为轨迹"的可解释性比"模型内部表征"的可解释性更直接有用：它不要求理解模型的黑盒，而是要求记录系统每一步做了什么、为什么做、结果是什么。从双平面架构的视角看，这恰好是确定性控制平面的核心产出——可审计、可回放的事件流。

12.4 公平性与资源分配（对应L2推理服务层与L5编排层）

当多个用户或多个Agent共享同一推理服务集群时，公平的资源分配成为伦理问题。传统操作系统通过cgroup（控制组）实现CPU、内存和I/O的公平分配[?]; 模型原生计算系统同样需要为每个Agent或任务设定上下文窗口预算、推理时间预算和成本预算。

具体问题包括：推理资源是否公平分配？大用户是否垄断了GPU资源？长任务的Agent是否占用了过多KV缓存，导致短任务饥饿？工具调用的副作用是否对所有用户一视同仁？这些公平性问题直接关联到公理5（最小权限公理）在资源管理层面的延伸。

13 结论与展望

本文的核心贡献可以概括为一个统一框架、一个架构解法和一组可计算原则。

13.1 核心贡献

统一框架：ICAM六层模型 本文提出智能计算体系结构模型（ICAM），将模型原生计算系统定义为六个功能层（L1-L6），每层有明确的接口契约、设计公理和度量指标。ICAM不是对经典计算机体系结构的机械复制，而是一套面向概率式执行的分层抽象。它的价值在于将AIOS[?]、MemGPT[?]、MCP[?]、PagedAttention[?]等分散工作统一到同一套分层模型中。

架构解法：双平面架构 本文揭示了"LLM是CPU还是OS"的隐喻冲突根源，提出双平面架构作为统一解答。概率执行平面（模型推理）对应"能做什么"，确定性控制平面（Agent运行时中的调度器、审批器、日志系统）对应"应该做什么"。这一分离使得ArbiterOS的"Probabilistic CPU"[?]、AIOS的"kernel"[?]和AgentOS的"reasoning kernel"[?]不再互相矛盾，而是同一系统中不同平面的不同视角。

可计算原则：三条量化定律 语义局部性定律（ $S = 1/((1-H) + H \cdot \alpha^{-1})$ ）、上下文预算定律（ $W_{\text{eff}} \leq C \cdot \beta(L)$ ）和Agent加速比定律（ $S_{\text{agent}} = 1/((1-F) + F/(N \cdot E))$ ）为模型原生计算系统提供了类似Amdahl定律和Roofline模型对传统体系结构那样的可计算原则。这些定律在形式上与Amdahl定律同构，其价值不在于数学创新，而在于为模型原生计算的设计空间提供了可计算的决策依据。实证检验表明，这些定律与已发表的实验数据定性一致，但仍需更系统的定量验证。

13.2 定位与边界

本文明确承认以下边界：

第一，这些类比并非严格同构。模型不是确定性CPU，语义检索不是精确寻址，长期记忆也不是无损分页，Agent运行时也不是成熟操作系统。但正是这些"不完全相同"，构成了未来研究最值得投入的空间：如何为语义系统设计新的ABI、如何为不确定推理设计新的OS、如何为长期状态设计新的虚拟内存、如何为多Agent协作设计新的提交协议。

第二，本文的独特贡献不在于第一次把大模型与计算机体系结构进行类比——Ge等[?]、L2MAC[?]、Mi等[?]已经覆盖了几条关键路线。也不在于第一次讨论KV缓存或记忆管理——MemGPT[?]与MemOS[?]已经把记忆子系统高度系统化。本文的贡献在于**第一次将这些互相独立、甚至互相冲突的类比整合为一套全栈模型原生计算架构**，包含统一的分层模型、双平面架构、层间接口契约、设计公理体系和可计算的量化定律。

第三，本文是技术畅想型综述（visionary survey），为未来模型原生计算架构提供研究框架，而非最终方案。ICAM的六层划分和三条定律的参数化形式都可能在实践中被修正和细化。本文的目的是提出正确的问题和可验证的预测，而非给出终极答案。

第四，本文的框架尚未充分建模人机协作的结构约束。工业实证显示，工程师在工作中约60%使用AI，但只能完全委托0-20%的任务[?]。这一"协作悖论"暗示L5-L6接口的任务提交契约需要增加"委托置信度"维度——反映任务可验证性和组织上下文依赖度——这超出了当前ICAM模型的范围。

13.3 未来方向

如果说过去十年的重点是"训练出更强的基础模型"，那么未来十年的一个核心问题，可能就是："如何把基础模型、上下文记忆、工具接口与Agent运行时组织成真正可编程、可扩展、可治理的模型原生计算体系结构。"

这个问题的答案可能来自多个方向的交汇：L2层的推理服务优化（KV缓存、量化、speculative decoding）使模型服务更高效；L3层的上下文编译和记忆管理使Agent拥有真正的工作记忆；L4层的协议标准化（MCP、A2A）使工具和Agent的互联从"点对点"进化为"总线式"；L5层的Agent OS使任务分解、权限控制和失败恢复从"手工编码"进化为"平台原生"；L6层的工作流自动化使最终用户可以用自然语言定义复杂任务。

经济层面的证据已经令人信服：TELUS创建了超过13,000个定制AI解决方案，节省了超

过500,000小时，工程代码交付速度提升30%；CRED通过将开发者转移到更高价值工作实现了执行速度翻倍；约27%的AI辅助工作由原本不会被执行的任务组成[?]。这些数据表明，模型原生计算不仅加速现有工作，更拓展了经济可行性的前沿。

本文为这个方向提供了一个系统化的研究框架：一个分层模型（ICAM）、一组设计公理、三条可计算定律和一个从短期到长期的研究路线图。希望这一框架能够帮助研究者和工程师在快速演化的模型原生计算生态中，找到系统层面的关键问题和可验证的设计原则。

Table 1: 计算机体系结构与大模型模型原生系统栈的类比映射。每行将一个经典计算机概念与模型原生计算中的对应物进行对照，并给出具体示例、核心相似性、本质差异与由此产生的研究方向

经典对象	模型原生计算对象	核心相似性与示例	本质差异	工程意义
ISA / ABI	提示模板、工具规范、Skill/Agent接口	都定义上层可见的调用契约与组合边界。例如：RISC-V的ECALL指令语义是精确定义的[?]; MCP的tools/call接口规范了参数schema和返回格式[?]	提示不是严格形式语义，存在歧义与漂移。同一提示词在不同模型版本上行为可能不同	需要稳定schema、版本号与回放兼容
CPU核心	基础模型前向计算	都是通用执行核心，把输入映射为输出。例如：CPU把机器指令序列映射为寄存器状态变化；模型把token序列映射为下一个token的概率分布[?, ?]	模型是概率式推断，不是确定性逻辑电路。同一输入可产生不同输出	应显式区分模型能力、推理策略与运行时
微架构	注意力内核、解码算法、服务内核	决定相同“接口”下的真实性能与成本。例如：超标量流水线vs. FlashAttention的IO感知分块计算[?]; 分支预测vs. 推测解码[?]	大模型“微架构”跨软件/硬件边界	联合优化FlashAttention、speculative decoding与并行策略
L1/L2缓存	会话级KV缓存、前缀缓存	都服务时序局部性与热点复用。例如：CPU L1缓存保存最近访问的缓存行；KV缓存保存最近计算的Key/Value向量，避免重复计算[?, ?]	语义缓存命中条件比地址缓存弱得多。地址相等是布尔判断，语义相似是连续值比较	需要块化分配、共享、失效与监控
主存/虚拟内存	上下文窗口、外部记忆、虚拟上下文	都通过分层制造“大空间”幻觉。例如：虚拟内存让进程以为拥有连续大地址空间；虚拟上下文管理让Agent以为拥有无限记忆[?]	语义摘要会损失信息，不是无损分页。OS页换入换出是逐字节精确的，摘要压缩不可逆	需要热/温/冷记忆分层、再检索与回填策略
操作系统	Agent运行时	都负责调度、权限、I/O与失败处理。例如：Linux的cgroup隔离进程资源[?]; Codex的sandbox隔离Agent的文件访问[?]	Agent自身含不确定推理与语言接口	需要沙箱、审批、子Agent、hooks与日志
系统调用/驱动	工具调用、MCP服务器	都把外部能力纳入统一访问接口。例如：open()系统调用把文件操作标准化；MCP的tools/call把工具调用标准化[?]	工具结果可能非确定、异步且副作用强	需要capability标注、鉴权、幂等设计
文件系统/磁盘	向量库、对象存储、知识库	都承载大容量、持久化状态。例如：ext4按inode组织文件；Milvus按向量索引组织知识[?]	语义检索不是精确寻址。find(path)返回唯一结果，向量搜索返回Top-K近似匹配	需引入provenance、版本快照和TTL
进程/线程	Agent / Sub-agent / Task graph	都是并发执行与隔离的基本单位。例如：POSIX线程共享地址空间但有独立栈；Codex子Agent共享项目上下文但独立执行[?]	共享语义状态比共享内存更脆弱。“理解同一份文档”不等于“拥有相同理解”	需要预算、优先级与结果合并

Table 2: ICAM各层与经典计算机体系结构的对应关系

ICAM层	经典对应	职责	核心差异	代表实现
L1	硬件/ISA	执行基本计算操作	概率式矩阵运算而非确定性逻辑门	GPU, TPU, 存算一体[?]
L2	微架构/缓存	管理计算资源与热点数据	KV缓存按语义序列增长而非地址寻址	vLLM, SGLang[?, ?]
L3	虚拟内存	扩大可寻址工作集	语义摘要有损而非无损分页	MemGPT, MemoryOS[?, ?]
L4	系统调用/ABI	统一外部能力访问接口	工具语义含自然语言歧义	MCP, A2A[?, ?]
L5	操作系统	调度、权限、容错	Agent自身含不确定推理	Codex, Claude Code[?, ?]
L6	用户应用	承载最终用户任务	任务可含开放式目标和模糊约束	SWE-agent, OpenHands[?, ?]

Table 3: 双平面与ICAM的交叉映射

ICAM层	概率执行平面的成分	确定性控制平面的成分
L1	矩阵运算、注意力计算（概率性输出）	精度校验、硬件错误检测
L2	KV缓存的语义匹配、前缀共享	缓存容量管理、回收策略
L3	语义检索、上下文编译、摘要生成	记忆保留策略、版本戳、状态持久化
L4	工具结果的语义理解	工具schema、协议规范、权限审批、审计
L5	Agent推理与决策（概率性）	任务调度、权限控制、失败恢复、trace记录
L6	两个平面的消费者：同时获得智能输出和可审计轨迹	

Table 4: 三条量化定律的实证数据汇总

	定律I（语义局部性）	定律II（上下文预算）	定律III（Agent加速比）
核心公式	$S = 1 / ((1 - H) + H / \alpha)$	$W_{\text{eff}} \leq C \cdot \beta(L)$	$S = 1 / ((1 - F) + F / (NE))$
经典对应	Amdahl定律 ($f \leftrightarrow H$, $p \leftrightarrow \alpha$)	Denning工作集模型 ($W(\tau) \leftrightarrow W_{\text{eff}}$)	Amdahl定律+ 编排开销 ($E < 1$)
支持数据	vLLM: 2-4×提升 ($H \approx 0.56-0.83$) [?]; Prompt Cache: 8-60× TTFT降低 ($H \approx 0.75-0.98$) [?]	RULER: 4K→128K时准确率从>95%降至<70%[?]; LongBench v2: 深度推理衰减更快[?]	AutoGen: Agent 2→8时时间减少收窄[?]; GTA-2: 开放工作流仅14.39%[?]
待验证	语义缓存的H分布	$\beta(L)$ 的精确函数形式	E与任务复杂度的定量关系